RESEARCH + ENGINEERING MASTER BLUEPRINT

NEXUS-OS

A Secure, Reliability-Aware Operating System for Autonomous AI Agents

Problem Statement • Architecture • Security • Kernel Model • Benchmarks • 12-Month Plan

Models propose. The kernel decides. Sandboxes execute. Evidence proves.

Prepared for a two-person systems + ML research team

Version 1.0 | 13 July 2026 | Working codename

IMPORTANT: NEXUS-OS is a temporary codename. Complete trademark, package and repository-name checks before public release.

Contents

A navigational outline of the master blueprint. Word heading styles are applied throughout for document navigation.

Document Control

Field	Value
Purpose	Establish a complete, research-backed plan for building an AI-native agent operating system and its compact control-plane model
Intended readers	Project founders, research mentors, systems engineers, ML researchers, security reviewers and potential contributors
Primary outcome	A buildable roadmap from a 10-12 week MVP to a publication-grade, open-source platform
Scope	Linux-hosted agent control plane, secure execution, scheduling, memory, reliability, observability, model routing and Kernel Model research
Out of scope	Replacing the Linux kernel, training a frontier general-purpose LLM, or providing unrestricted autonomous access to production systems
Decision rule	Ship measurable mechanisms and benchmarks; avoid features that cannot be evaluated or defended technically

Executive Summary

NEXUS-OS is an AI-native control plane for running autonomous software agents as governed computational workloads. It treats agents as first-class processes with explicit identity, lifecycle state, resource budgets, capabilities, context, memory, provenance, reliability estimates and parent-child relationships. Rather than allowing a language model to call tools directly, the system converts each proposed action into a typed request that passes through deterministic schema validation, taint analysis, policy evaluation, risk scoring, budget checks, human-approval gates and an isolation boundary before execution.

The project occupies the space between an operating system, a distributed runtime, a zero-trust security platform and a reliability controller for AI. It is intentionally not a conventional kernel, not a multi-agent prompt library and not a thin wrapper around commercial models. Its strongest contribution is a set of enforceable abstractions for autonomous computation: `AgentProcess`, `ActionProposal`, `Capability`, `Budget`, `ContextPage`, `MemoryObject`, `Checkpoint`, `EvidenceBundle` and `AgentWorkload`.

The platform will support local and cloud models, Model Context Protocol (MCP) tools, Agent-to-Agent (A2A) communication, sandboxed command execution, event-sourced replay, policy-as-code, OpenTelemetry-compatible observability, memory paging and reliability-aware routing. A later research phase adds a compact **Kernel Model** trained on agent trajectories. This model predicts failure risk, resource needs, model choice, context utility, recovery actions and scheduling priority. Crucially, the Kernel Model remains advisory: it can recommend, but it cannot create permissions, override policy or execute tools independently.

The build is organized as an incremental programme. A two-person team can produce a compelling runtime MVP in 10-12 weeks, a security-and-replay release in roughly five months, a reliability-aware

research platform in eight months and a compact Kernel Model plus benchmark release within a year. Each phase produces a standalone demonstration and an independently valuable open-source artifact.

What makes the project exceptional

- It combines operating-systems thinking with modern agent infrastructure instead of merely orchestrating prompts.
- It makes security, reliability and reproducibility architectural primitives rather than optional middleware.
- It creates a defensible dataset of agent trajectories, policy decisions, resource traces and failure labels.
- It offers multiple publication paths: scheduling, context paging, failure prediction, secure tool use, routing and reproducible agent evaluation.
- It is ambitious enough to be memorable while remaining decomposable into working releases.

Target end state

A user should be able to submit a high-level workload such as:

TEXT

```
Inspect this repository, determine why CI fails, prepare a minimal patch,
run tests, generate an evidence report, and open a draft pull request.
```

NEXUS-OS should then spawn governed agents, allocate budgets, choose models, mediate tools, isolate execution, pause for approval where needed, detect uncertainty or unsafe behavior, retry recoverably, collect evidence and produce a replayable trace of the entire run.

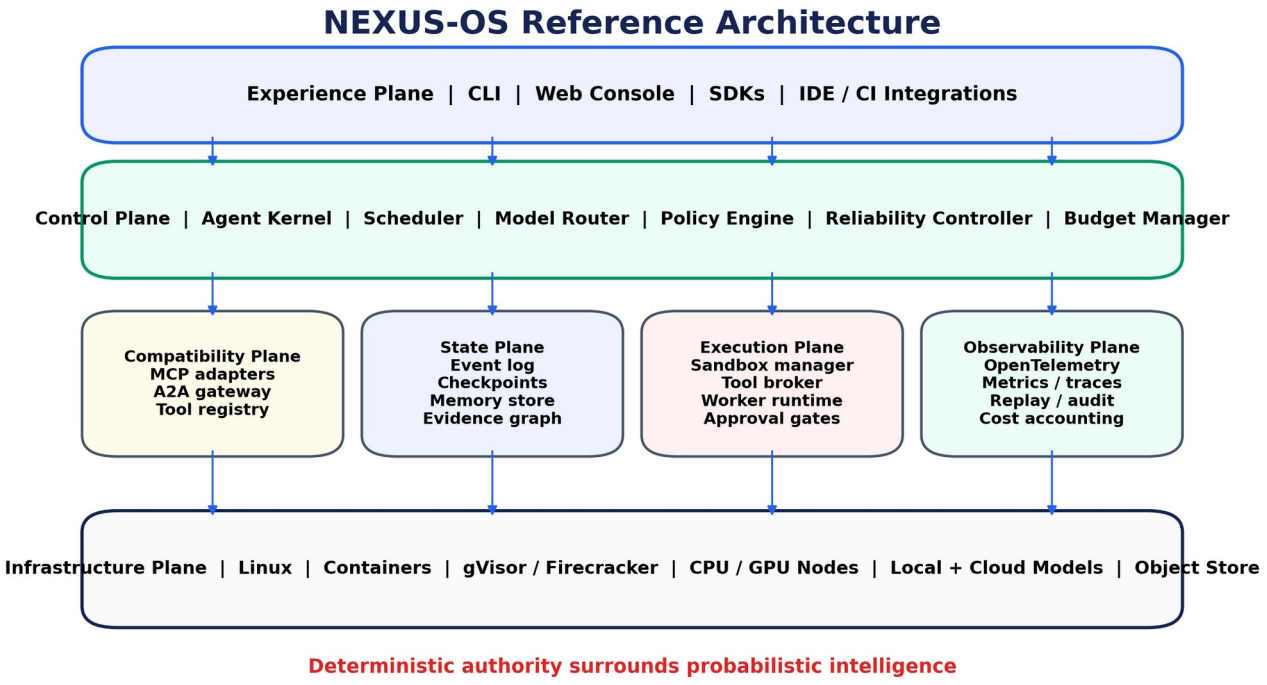

Figure 1. Reference architecture.

Part I - Problem, Opportunity and Product Definition

1. Problem Statement

Autonomous AI agents are increasingly capable of browsing, editing files, invoking APIs, executing code, purchasing services and coordinating other agents. Most existing frameworks optimize for connecting models to tools and composing workflows. They generally assume that the model is a trusted planner and that application code can safely translate model output into side effects.

That assumption breaks down in realistic deployments. Language models are probabilistic, vulnerable to prompt injection, inconsistent under distribution shift and often poorly calibrated about failure. Tool outputs can contain malicious instructions. Retrieved memories can be poisoned. Credentials may be overprivileged. Multi-agent delegation can obscure accountability. Long-running tasks consume unbounded tokens, money and compute. When an agent fails, developers often lack a causal record of which observation, model response, policy decision or tool action produced the outcome.

The engineering problem is therefore not simply “how do we give an LLM tools?” It is:

> **How can autonomous AI workloads be executed with enforceable least privilege, bounded resources, measurable reliability, recoverable state and complete evidence, while remaining portable across models, tools and deployment environments?**

1.1 Root causes

1. **No trusted control boundary.** Model-generated actions frequently flow directly into Python functions, shell commands or browser automation.
2. **Ambient authority.** Agents inherit broad filesystem, network and secret access from the host process.
3. **Weak lifecycle semantics.** “Agent,” “task,” “run” and “message” are often library-specific objects without standard admission, suspension, termination or recovery behavior.
4. **Unbounded budgets.** Token, monetary, latency, compute and risk limits are tracked informally or after the fact.
5. **Memory without integrity.** Vector stores optimize semantic retrieval but rarely express provenance, trust, conflict, expiry, taint or access control.
6. **Opaque failures.** Logs show events but do not preserve causality, state checkpoints, policy reasoning or reproducible inputs.
7. **Model lock-in.** Control logic is entangled with a provider’s message format, tool schema and model-specific behavior.
8. **Benchmark fragmentation.** Agent success, security, cost and reproducibility are evaluated separately, making trade-offs difficult to compare.

1.2 Consequences

Without an operating layer, organizations face data exfiltration, destructive actions, policy violations, runaway expenditure, unverifiable results, approval fatigue and fragile integrations. Developers

compensate by inserting ad hoc guardrails, hard-coded allowlists and retries throughout application code. This creates a system that is difficult to audit and nearly impossible to reason about formally.

1.3 Proposed solution

NEXUS-OS introduces a deterministic agent kernel above Linux. Every autonomous workload is admitted through a declared manifest. Every agent has an identity, capabilities and budgets. Every external action becomes a typed proposal. Every side effect is mediated by policy and executed in an appropriate sandbox. Every state transition is recorded in an append-only event log. Every important artifact has provenance. Every failure can be inspected and, where possible, replayed from a checkpoint.

2. Vision, Mission and Design Position

2.1 Vision

Make autonomous computation governable in the same way conventional operating systems made general-purpose computation manageable: through explicit abstractions, resource control, isolation, scheduling and durable interfaces.

2.2 Mission

Build an open, model-agnostic platform that allows developers and institutions to run capable AI agents with strong security boundaries, reliability controls, transparent costs and reproducible evidence.

2.3 Product position

NEXUS-OS should be described as:

> **A secure, reliability-aware operating system and distributed control plane for autonomous AI agents.**

It runs above an existing host operating system and uses Linux primitives, containers, microVMs and remote workers as execution substrates. “Operating system” refers to the abstractions and resource-governance role, not to booting bare metal.

2.4 Non-goals

- Reimplementing Linux, device drivers, virtual memory or a complete POSIX userspace.
- Training a general-purpose frontier model comparable to DeepSeek, GLM or GPT-class systems.
- Building another visual prompt-flow builder with no systems contribution.
- Guaranteeing that an LLM is truthful or safe in all circumstances.
- Permitting fully autonomous high-impact actions without policy or human oversight.
- Supporting every model provider, tool protocol and workload in the first release.

3. Why Now

The agent ecosystem is converging on several necessary pieces: standard tool protocols, agent-to-agent communication, sandboxed code execution, identity, observability and policy. Academic systems such

as AIOS and MemGPT demonstrate that OS-inspired scheduling and memory abstractions are meaningful. Industry platforms now expose agent SDKs, sandboxing, identity and enterprise governance. MCP and A2A are becoming interoperability layers, while OpenTelemetry and policy-as-code provide mature foundations for visibility and enforcement.

The strategic gap is that these pieces remain fragmented. A developer can choose an orchestration framework, a sandbox, an observability backend and a policy engine, but there is no broadly adopted, model-independent control plane that unifies lifecycle, budgets, capabilities, reliability, memory integrity and causal replay. This is precisely the layer NEXUS-OS targets.

4. Success Criteria

The project is successful only if it produces evidence beyond a polished demo.

Dimension	Success criterion
Functionality	A non-trivial coding or research workload completes through governed multi-step execution
Security	Unauthorized file, network, secret and shell actions are denied and logged
Reliability	The controller reduces catastrophic failures or unnecessary cost versus a direct-agent baseline
Reproducibility	A run can be replayed from a checkpoint with identical tool inputs and attributable divergence
Efficiency	Routing and scheduling reduce cost or latency without unacceptable success-rate loss
Portability	At least three model backends and two tool protocols work through stable interfaces
Research	At least one hypothesis is evaluated with baselines, ablations and statistically defensible results
Open source	Installation, examples, threat model, benchmark harness and contribution guide are public
Demo quality	A live dashboard visibly explains what the system is doing and why

5. Flagship Use Cases

5.1 Governed coding agent

Inspect a repository, diagnose failing tests, edit code, run commands and prepare a patch. The agent receives repository-scoped write access, package-install restrictions, a network allowlist and a bounded execution budget. Each patch is linked to tests, commands and model decisions.

5.2 Secure research agent

Search approved sources, download papers, extract claims, maintain citations and draft a report. Retrieved content is treated as untrusted; prompt-injection strings cannot directly trigger tools. Each claim is linked to evidence and freshness metadata.

5.3 Data-analysis agent

Load a dataset, write analysis code, execute it in an isolated environment and produce tables or plots. Privacy policy controls which columns may be sent to cloud models. Computation can be routed locally when data sensitivity is high.

5.4 Incident-response assistant

Collect logs, propose diagnostic commands and coordinate specialist agents. Read-only access is the default. High-impact remediation requires explicit approval and a rollback plan.

5.5 Edge-cloud autonomous workflow

Run lightweight planning and privacy-sensitive tasks locally, escalating difficult reasoning to a remote model only when policy, connectivity, cost and reliability permit.

Part II - Landscape, Standards and Strategic Gap

6. Research and Industry Landscape

6.1 OS-inspired agent systems

The AIOS research line frames large language model agents as operating-system workloads and introduces modules for scheduling, context management, memory, storage and tool execution. It validates the central intuition that agent systems benefit from explicit resource governance. MemGPT similarly uses hierarchical memory and virtual-memory analogies to extend agents beyond a fixed context window. These systems are important intellectual foundations, but the opportunity remains to integrate security policy, deterministic authority, causal replay and reliability-aware control into a coherent implementation.

Recent work on agent scheduling shows that asynchronous, tool-using workloads differ from conventional batch inference. Agents alternate between model calls, external I/O and local execution; a scheduler that ignores these phases can underutilize resources or create head-of-line blocking. Security surveys now catalogue dozens of attack families across models, tools, memory, communication and infrastructure. Memory research is also moving from simple vector retrieval toward agent-native memory with explicit operations, organization and lifecycle.

6.2 Industry movement

Major vendors are independently building elements of the control-plane stack:

- Microsoft's agentic Windows direction emphasizes agent identity, isolated workspaces and policy-controlled capabilities.
- NVIDIA's NemoClaw and OpenShell direction combines sandboxing, privacy controls and routing between local and cloud models.
- OpenAI's evolving Agents SDK includes managed tools, tracing and sandbox-oriented execution capabilities.
- Google's Agent Development Kit provides multi-agent composition, evaluation and deployment integrations.

These developments confirm the problem is real. They also mean NEXUS-OS must differentiate through openness, model neutrality, enforceable least privilege, explicit lifecycle semantics, benchmarked reliability and a research-grade trajectory dataset rather than competing on the number of integrations.

6.3 Adjacent frameworks

Category	Examples	Strength	Gap NEXUS-OS addresses
Agent orchestration	LangGraph, AutoGen, CrewAI, Semantic Kernel	Workflow composition and developer ergonomics	Kernel-level authority, resource governance, isolation and replay are not the primary abstraction
Coding-agent platforms	OpenHands, SWE-agent and commercial coding agents	Strong task-specific loops and tool usage	Generalized capabilities, budgets, policy and multi-workload runtime
Tool protocols	MCP	Portable discovery and invocation of tools	Protocol does not itself enforce least privilege, budgets or sandbox strength
Agent interoperability	A2A	Cross-agent task exchange	Requires local identity, trust, authorization and evidence semantics
Sandboxes	Docker, gVisor, Firecracker, WASI	Stronger execution isolation	Do not understand agent intent, provenance, memory or model risk
Policy	OPA/Rego, Cedar-like systems	Deterministic authorization	Need agent-specific inputs, taint, risk and approval integration
Observability	OpenTelemetry	Portable metrics, logs and traces	Needs agent event taxonomy, causal replay and evidence bundles
Model serving	vLLM, SGLang, Ollama	Efficient model execution	Does not decide which model should handle which governed action

7. Standards Strategy

NEXUS-OS should avoid inventing protocols where stable ecosystems already exist.

7.1 MCP for tools and context

MCP adapters should translate external tool descriptions into an internal canonical tool schema. The Tool Broker remains the enforcement point. A connected MCP server is never implicitly trusted; each

advertised tool receives an administrator-defined capability mapping, argument constraints, data classification and sandbox profile.

7.2 A2A for cross-agent delegation

An A2A gateway can support delegation across runtimes. External tasks must be mapped to local identities and scoped capabilities. Evidence and attestation should accompany delegated results so that a receiving agent knows which system executed an action, under what policy and with which artifacts.

7.3 OpenTelemetry for observability

Use trace IDs to connect model calls, policy decisions, tool invocations, sandbox events and artifact creation. Define semantic conventions for agent-specific spans rather than building a proprietary telemetry format. Sensitive content must be redacted or referenced by hash before export.

7.4 OPA/Rego for policy-as-code

OPA provides deterministic policy decisions with explainable inputs and outputs. NEXUS-OS should add agent-specific data such as task purpose, capability grants, taint labels, resource budgets, risk estimates, user identity and proposed side effects.

7.5 SPIFFE/SPIRE for workload identity

For distributed deployments, issue short-lived workload identities to workers and services. Avoid static shared API keys between the control plane, executors and model gateways.

7.6 Supply-chain standards

Use signed container images, Software Bills of Materials, SLSA-oriented build provenance, in-toto attestations and Sigstore-compatible signing for released binaries, benchmark images and model artifacts.

8. Strategic Gap and Project Wedge

The project should not initially claim to be a universal agent platform. Its first wedge is:

> **The safest and most inspectable way to run coding and research agents across local and cloud models.**

This wedge is appropriate because coding and research tasks:

- Exercise files, shell, network, memory and model routing.
- Have measurable outcomes such as passing tests or supported citations.
- Produce meaningful security risks without requiring physical-world autonomy.
- Generate rich trajectories for the Kernel Model.
- Support public benchmarks and reproducible environments.

After establishing the core, the runtime can expand to enterprise operations, data analysis, cyber workflows and edge-cloud autonomy.

Part III - Core Abstractions and Invariants

9. Core Abstractions

9.1 AgentWorkload

A declarative specification for a complete user request. It contains the objective, expected outputs, initial data, model constraints, capabilities, budgets, policy profile, approval rules, benchmark metadata and success criteria.

9.2 AgentProcess

A schedulable, isolated reasoning entity. It is not synonymous with an OS process; it may span model calls and tool executions while maintaining a persistent logical identity.

TEXT

```
AgentProcess
├── process_id and workload_id
├── principal / identity
├── objective and current subgoal
├── parent and child process IDs
├── lifecycle state and priority
├── model policy and current backend
├── capabilities and data labels
├── token, money, time, compute and risk budgets
├── active context-page set
├── memory namespace and evidence graph
├── reliability state
└── checkpoint and event-log offsets
```

9.3 TaskNode

A node in a typed task graph. Nodes declare dependencies, estimated resources, side-effect class, success predicate and compensation behavior. The graph can evolve, but every mutation is recorded.

9.4 ActionProposal

A structured request generated by an agent or planner. It includes action type, target tool, arguments, rationale, expected effect, data dependencies, confidence, reversibility and requested capability. Free-form text is never executed directly.

9.5 Capability

A non-transferable or explicitly delegable grant authorizing a narrow class of action. Capabilities are scoped by resource, operation, constraints, expiry and delegation depth.

Examples:

- Read `/workspace/repo/**` but not `.env` files.
- Write only under `/workspace/repo/src/**`.
- Connect to `github.com` over HTTPS.
- Run `pytest` for at most 15 minutes with no privileged syscalls.
- Use a named secret only through a brokered API call; never reveal its plaintext.

9.6 Budget

A first-class resource contract. Budgets can apply to a workload, process, task node, model or tool.

Budget type	Example control
Token	Maximum input, output and reasoning tokens
Monetary	Provider spend ceiling and per-action limit
Wall-clock	Deadline and maximum runtime
Compute	CPU time, memory, GPU memory and accelerator seconds
Tool	Invocation count, rate limit and cumulative execution time
Network	Allowed bytes, destinations and request rate
Risk	Maximum cumulative risk score or number of high-risk actions
Human attention	Maximum approval prompts and escalation frequency
Energy, optional	Estimated joules or carbon-intensity-aware routing

9.7 ContextPage

A bounded unit of context with content, token cost, provenance, trust, taint labels, access policy, expiry, retrieval score and causal links. Pages can be active, compressed, archived, invalidated or quarantined.

9.8 MemoryObject

A durable object such as a fact, event, plan, preference, code artifact or episodic summary. MemoryObjects preserve source references and can express contradiction, supersession and confidence.

9.9 Checkpoint

A recoverable snapshot of logical agent state: task graph, context-page references, memory version, sandbox filesystem snapshot, budgets, policy version and event-log offset. Model hidden state need not be captured initially; deterministic inputs and outputs are sufficient for operational replay.

9.10 EvidenceBundle

A signed or hash-linked package containing inputs, policy decisions, model metadata, tool invocations, outputs, artifacts, tests, approvals and final claims. It is the unit delivered to users, auditors or downstream agents.

10. Design Principles

1. **Deterministic authority, probabilistic advice.** Models propose; trusted code decides and executes.

2. **Least privilege by construction.** A process starts with no capabilities beyond explicitly granted ones.
3. **Everything expensive or dangerous is budgeted.** Cost, time, compute, risk and human attention have ceilings.
4. **Untrusted by default.** Web pages, repository files, emails, tool output and agent messages may contain hostile instructions.
5. **Typed mediation.** Side effects cross a schema-validated interface, never an unparsed natural-language boundary.
6. **Event sourcing.** State changes are derived from immutable events and versioned policies.
7. **Causal evidence.** Artifacts and claims link back to observations, actions and policy decisions.
8. **Safe degradation.** When confidence falls or infrastructure fails, the system pauses, reduces privilege, escalates or exits cleanly.
9. **Model neutrality.** Control semantics do not depend on one provider's message format.
10. **Researchability.** Every mechanism has a baseline, metric, ablation and exportable dataset.

11. Security and Reliability Invariants

The initial implementation should encode the following as testable invariants:

- No tool can be invoked without a valid process identity and capability.
- No capability can be created by model output.
- No secret plaintext is inserted into an LLM prompt unless policy explicitly permits it.
- No network destination is reachable merely because a shell process can resolve it.
- Every side-effecting action has a unique proposal ID and policy decision.
- Every budget debit is atomic and attributable.
- A denied action produces no partial external side effect.
- High-risk actions require a declared compensation plan or human approval.
- Retrieved content cannot change system policy.
- A process cannot read another process's memory namespace without a grant.
- Policy versions, model versions and tool versions are included in replay data.
- Kernel Model recommendations cannot bypass deterministic checks.

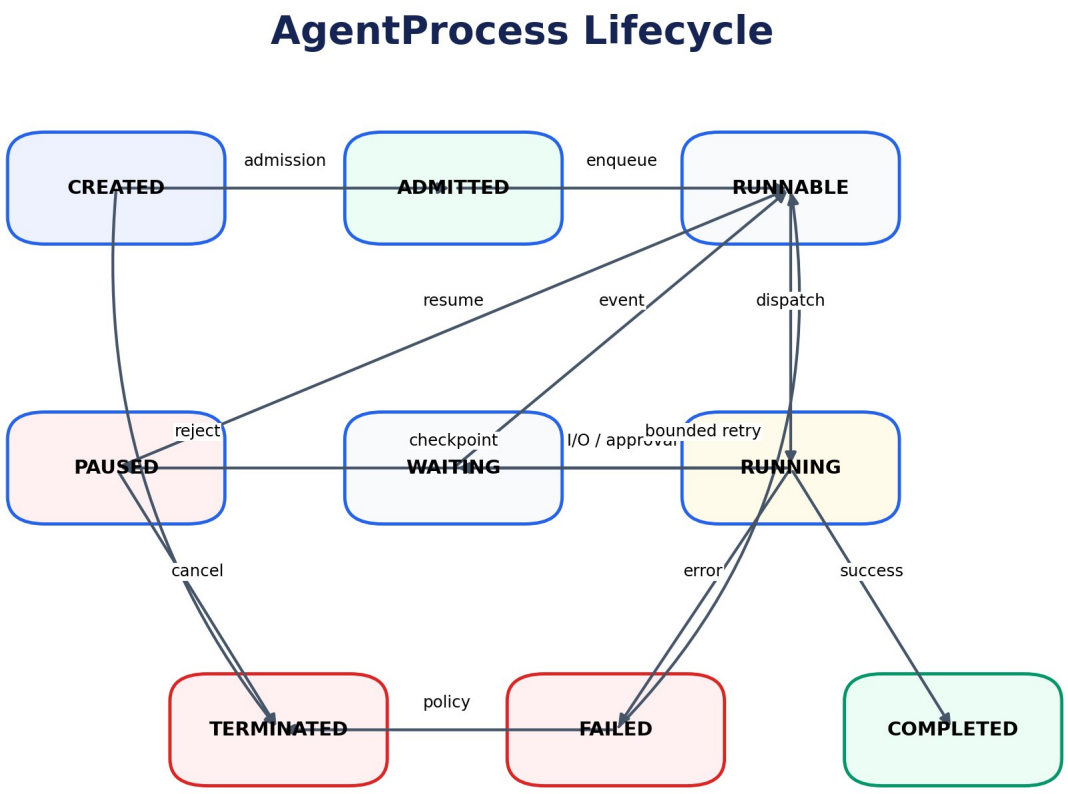

Figure 2. Agent lifecycle.

12. Lifecycle Semantics

An `AgentProcess` transitions through `CREATED`, `ADMITTED`, `RUNNABLE`, `RUNNING`, `WAITING`, `PAUSED`, `COMPLETED`, `FAILED` and `TERMINATED` states. Admission validates the workload manifest, resolves policy, estimates resources and confirms an execution substrate. Waiting is explicit for model calls, tool I/O, approvals or dependencies. Pausing creates a checkpoint when possible. Retries are bounded, classified and linked to the original failure.

A child agent inherits only capabilities explicitly marked as delegable, optionally narrowed by the parent. Budgets are reserved rather than copied. Terminating a parent should apply a documented policy to children: cascade, orphan under a supervisor, or allow bounded completion.

Part IV - Reference Architecture and Component Design

13. Architecture Overview

NEXUS-OS is divided into six logical planes. The separation prevents frontend, model-provider or execution-specific concerns from leaking into core policy.

13.1 Experience plane

- Command-line interface for running, inspecting, pausing, approving and replaying workloads.
- Web console displaying process trees, task graphs, budgets, model routes, policy decisions, telemetry, artifacts and security alerts.
- Python and TypeScript SDKs.
- IDE, CI and Git hosting integrations.

13.2 Control plane

- Agent Kernel and lifecycle manager.
- Admission controller.
- Task-graph manager.
- Scheduler and resource allocator.
- Model router and provider gateway.
- Capability manager and policy decision point.
- Reliability controller.
- Budget manager.
- Approval coordinator.

13.3 Compatibility plane

- MCP client and server adapters.
- A2A gateway.
- Tool registry and canonical schema translation.
- Agent-framework adapters for selected ecosystems.
- Model-provider normalization.

13.4 State plane

- Append-only event store.
- Materialized current-state views.
- Checkpoint metadata and filesystem snapshots.
- Context and memory stores.
- Artifact object store.
- Evidence and provenance graph.

13.5 Execution plane

- Sandboxed worker runtime.
- Tool Broker.
- Shell, browser, code-execution and API adapters.
- Network proxy and secret broker.
- Local and remote executors.
- Snapshot, restore and compensation service.

13.6 Observability plane

- OpenTelemetry traces, logs and metrics.
- Cost accounting and budget ledgers.
- Security decision records.

- Reliability and calibration dashboards.
- Replay engine and divergence viewer.

14. Agent Kernel

The Agent Kernel is the trusted coordinator. It should remain small enough to audit and should not include business-specific prompts.

14.1 Responsibilities

- Validate and admit workloads.
- Assign process identities.
- Maintain lifecycle state.
- Enforce capability and budget invariants.
- Coordinate scheduling and execution.
- Persist events before acknowledging state changes.
- Invoke policy and reliability services.
- Handle cancellation, timeouts, retries and recovery.
- Produce the final EvidenceBundle.

14.2 What does not belong in the kernel

- Tool-specific implementation logic.
- Long prompts and domain-specific agent personalities.
- Provider-specific response parsing beyond normalized adapters.
- Vector-search algorithms.
- UI state.
- Untrusted plugin code.

14.3 Recommended implementation

Use Rust for the kernel and critical services because ownership, strong typing, concurrency safety and predictable performance suit a security-sensitive runtime. Tokio can support asynchronous model and tool I/O. Axum can expose HTTP APIs and Tonic can support internal gRPC. Python remains the primary agent and ML SDK language.

15. Admission Controller

Admission is analogous to validating and scheduling a job in a cluster.

15.1 Admission checks

1. Parse the workload manifest and validate its schema.
2. Resolve user and tenant identity.
3. Load the referenced policy bundle.
4. Validate model, tool and sandbox availability.
5. Estimate minimum and maximum resources.
6. Check capability conflicts and forbidden combinations.
7. Verify budget ceilings and provider quotas.

8. Classify data sensitivity and required locality.
9. Determine whether mandatory approvals are configured.
10. Create the root process and initial event.

Admission can return `accepted`, `accepted_with_restrictions`, `pending_approval` or `rejected` with machine-readable reasons.

16. Task Graph and Planner Boundary

A planner may create or modify a directed task graph, but graph mutations are proposals subject to validation. Each node declares:

- Input artifacts and data classifications.
- Expected output type and success predicate.
- Required capability class.
- Side-effect and reversibility category.
- Resource estimate and deadline.
- Dependencies and concurrency constraints.
- Retry and compensation policy.

The first release can use a simple model-driven planner with deterministic graph validation. Later research can compare hierarchical planning, search, learned policies and multi-agent decomposition.

17. Reliability-Aware Scheduler

Traditional schedulers optimize CPU fairness or throughput. Agent scheduling must consider alternating model, tool and waiting phases; model availability; deadlines; uncertainty; cost; risk; and expected value.

17.1 Scheduling inputs

- Process priority, age and deadline.
- Runnable task nodes and dependencies.
- Estimated model and tool latency.
- Token and monetary budget remaining.
- CPU, RAM, GPU and sandbox capacity.
- Reliability score and recent failure history.
- Action risk class.
- Human-approval availability.
- Data locality and privacy constraints.
- Fairness across users or tenants.

17.2 Baseline policies

1. FIFO.
2. Round robin by process.
3. Earliest deadline first.
4. Shortest expected action first.
5. Cost-aware routing with fixed priorities.

6. Work-conserving asynchronous scheduling.

17.3 Utility formulation

A practical policy can rank a candidate action `a`, model `m` and sandbox `s` with:

```text
U(a,m,s) = P_success(a,m) * V(a)
           - lambda_cost * E[cost]
           - lambda_latency * E[latency]
           - lambda_risk * E[risk]
           - lambda_deadline * deadline_penalty
           - lambda_human * approval_cost
```

Hard policy constraints are applied before optimization. The scheduler cannot trade away a security rule for higher utility.

17.4 Scheduling research opportunity

Compare heuristic scheduling with contextual bandits, constrained reinforcement learning and the Kernel Model. Evaluate task success, throughput, deadline misses, cost, tail latency, fairness and risk incidents under mixed workloads.

18. Budget Manager

Budgets are represented as hierarchical ledgers. A workload owns the top-level allocation. Child processes reserve portions and return unused capacity. Every debit includes an event ID and resource source.

18.1 Required behaviors

- Atomic reservation and debit.
- Soft-warning and hard-stop thresholds.
- Per-model and per-tool sublimits.
- Budget inheritance and delegation rules.
- Predictive checks before expensive actions.
- Emergency reserve for cleanup or evidence generation.
- Human-approved budget increases.
- Cost reconciliation against provider invoices.

18.2 Budget exhaustion actions

Depending on policy, the controller can compress context, switch models, reduce concurrency, skip optional tasks, request approval, checkpoint and pause, or terminate with a partial EvidenceBundle.

19. Model Gateway and Router

The Model Gateway normalizes provider APIs into a canonical interface and isolates credentials. The router chooses a backend per call rather than binding an entire workload to one model.

19.1 Routing dimensions

- Capability: reasoning, coding, vision, tool use and context length.

- Reliability on the current task class.
- Privacy and residency.
- Latency and availability.
- Input and output price.
- Local GPU capacity.
- Prompt and tool compatibility.
- Calibration and recent drift.
- Energy or carbon constraints, optionally.

19.2 Routing sequence

1. Filter models that violate policy or cannot support the request.
2. Estimate success, latency and cost.
3. Apply minimum reliability thresholds.
4. Select the lowest-cost model that meets constraints, or optimize utility.
5. Observe outcome and update online statistics.
6. Escalate when uncertainty, failure or risk crosses a threshold.

19.3 Required adapters for MVP

- One local backend through Ollama, vLLM or SGLang.
- One OpenAI-compatible remote endpoint.
- One second provider or open model for cross-model evaluation.
- Deterministic mock model for testing.

20. Tool Broker

The Tool Broker is the only path from agent proposals to external effects.

20.1 Tool registration

Every tool has:

- Stable tool ID and version.
- Typed input/output schema.
- Capability mapping.
- Side-effect class.
- Data classifications accepted and returned.
- Network and filesystem requirements.
- Default sandbox profile.
- Timeout and resource limits.
- Idempotency and compensation metadata.
- Redaction and logging rules.

20.2 Side-effect classes

Class	Meaning	Default handling
0	Pure computation	Execute in constrained sandbox

Class	Meaning	Default handling
1	Read-only external access	Allow with destination and data checks
2	Reversible local mutation	Snapshot or record inverse operation
3	External mutation with compensation	Require compensation metadata; approval by policy
4	Irreversible or high impact	Human approval and strongest isolation

20.3 Canonical action envelope

JSON

```
{
  "proposal_id": "ap_01J...",
  "process_id": "proc_01J...",
  "tool_id": "shell.exec/v1",
  "arguments": {"argv": ["pytest", "-q"]},
  "purpose": "Validate the proposed patch",
  "expected_effect": "Read repository files and create test cache files",
  "data_dependencies": ["artifact:patch-17"],
  "requested_capability": "cap:test-workspace",
  "reversibility": "snapshot-restorable",
  "model_confidence": 0.74
}
```

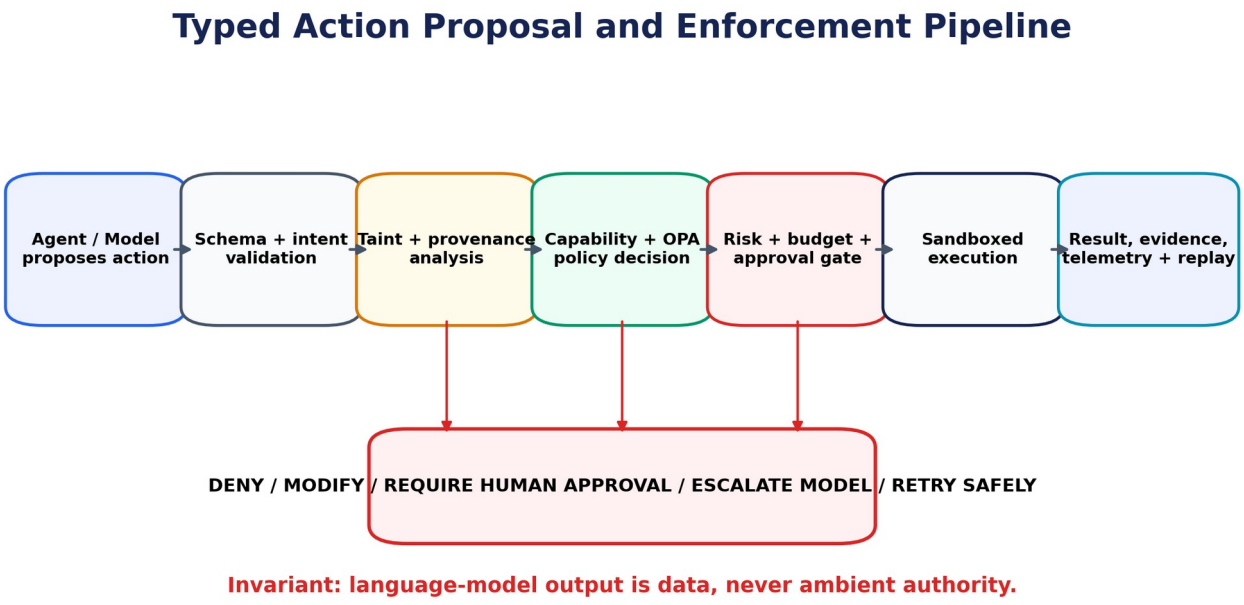

Figure 3. Policy and action flow.

21. Policy Engine and Capability System

Capabilities define what could be permitted; policy determines whether a specific proposal is permitted now.

21.1 Capability manifest example

YAML

```
capabilities:
  - id: repo-read
    resource: file:///workspace/repo/**
    operations: [read, stat, list]
    deny_patterns: ["**/.env", "**/id_rsa", "**/secrets/**"]

  - id: source-write
    resource: file:///workspace/repo/src/**
    operations: [create, write, rename, delete]
    constraints:
      max_changed_files: 12
      snapshot_required: true

  - id: test-runner
    resource: tool://shell.exec
    operations: [invoke]
    constraints:
      command_allowlist: [pytest, python, git]
      network: deny
      max_duration_seconds: 900
      max_memory_mb: 4096

  - id: github-read
    resource: https://github.com/**
    operations: [http_get]
    constraints:
      max_response_mb: 20
```

21.2 Policy input

JSON

```
{
  "subject": {"tenant": "demo", "process": "proc_42"},
  "workload": {"purpose": "fix_ci", "data_class": "internal"},
  "proposal": {"tool": "shell.exec", "side_effect": 2},
  "capabilities": ["test-runner"],
  "taint": ["untrusted_repository_text"],
  "budgets": {"risk_remaining": 12, "money_remaining": 4.10},
  "reliability": {"failure_probability": 0.18},
  "environment": {"sandbox": "gvisor", "network_mode": "deny"}
}
```

21.3 Policy decision

Return `allow`, `deny`, `modify`, `require_approval` or `require_stronger_sandbox`, along with reason codes, policy version and any rewritten constraints.

22. Approval Coordinator

Human approval is a scarce resource and should not become a substitute for engineering.

Approval requests must state:

- The exact proposed side effect.
- Why it is needed.
- Which data and capability are involved.
- Predicted risk and reversibility.
- A concise diff or preview.
- Alternatives the agent considered.
- Expiry time and whether a scoped recurring grant is possible.

Batch similar low-risk actions and avoid repeated prompts. Measure approval precision: the fraction of requests that humans judge genuinely necessary.

23. Distributed Worker Runtime

Workers execute approved actions and report signed results. A worker should have no authority beyond the task lease it receives.

23.1 Worker lease

- Workload and process identity.
- Tool and argument digest.
- Capabilities narrowed to one action.
- Sandbox profile.
- Resource limits.
- Network policy.
- Expiry and nonce.
- Artifact upload credentials scoped to a prefix.

23.2 Queue architecture

Begin with PostgreSQL transactions and an outbox table to avoid premature distributed complexity. Introduce NATS or another message bus when multiple workers, backpressure and remote execution become necessary.

Part V - Security Architecture

24. Threat Model

NEXUS-OS assumes that models, retrieved content, repository text, websites, tool output, remote agents and plugins may be compromised or adversarial. The trusted computing base should be limited to the kernel, policy engine, identity service, secret broker, worker launcher and selected sandbox components.

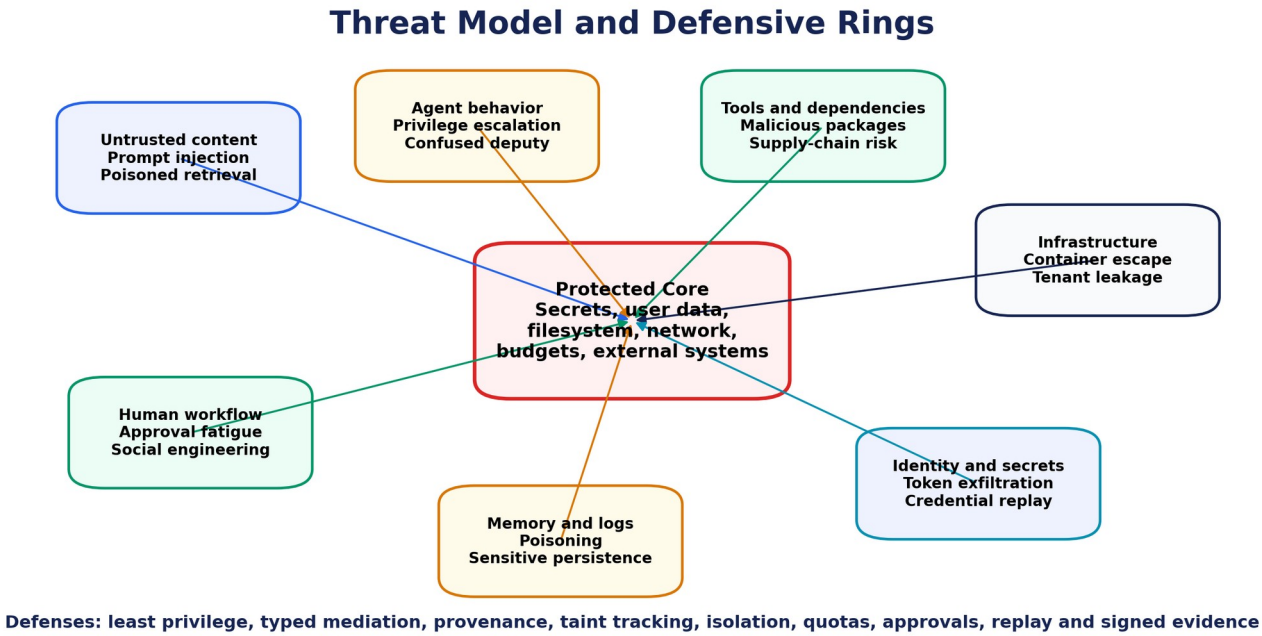

Figure 4. Threat model.

24.1 Assets to protect

- User files, source code and private datasets.
- Credentials, API tokens and signing keys.
- Network access and external services.
- Compute and monetary budgets.
- Model prompts, memory and conversation history.
- Integrity of produced artifacts and evidence.
- Tenant isolation.
- Availability of the runtime.
- Human decision channels.

24.2 Adversaries

- Malicious content author embedding prompt injection.
- Compromised MCP server or tool plugin.
- Malicious dependency or container image.
- External agent sending deceptive tasks or results.
- Curious or compromised model provider.
- Tenant attempting cross-namespace access.
- Insider misconfiguring policies or grants.
- Agent behavior that is harmful without malicious intent.

24.3 Attack categories

1. Direct and indirect prompt injection.
2. Tool-result manipulation and confused-deputy attacks.
3. Capability escalation or delegation abuse.
4. Secret exfiltration through prompts, logs, DNS or network requests.
5. Filesystem escape, container escape and host compromise.
6. Memory poisoning, stale facts and malicious summaries.

7. Cross-agent message spoofing or replay.
8. Denial of wallet through tokens, cloud APIs or compute.
9. Approval phishing and fatigue.
10. Supply-chain compromise.
11. Evidence tampering or selective logging.
12. Model endpoint substitution and response forgery.

25. Defense-in-Depth Controls

25.1 Identity and authentication

- Unique process and workload identities.
- Short-lived worker credentials.
- Mutual TLS between control-plane services.
- SPIFFE-compatible service identities in distributed deployments.
- Signed A2A messages with replay protection.
- Separate user, service, process and tool principals.

25.2 Authorization

- Default deny.
- Narrow, typed capabilities.
- Explicit delegation depth.
- Policy-as-code decisions for every side effect.
- Deny rules that cannot be weakened by workload manifests.
- Capability revocation and expiry.
- Two-person or human approval for selected high-impact classes.

25.3 Taint and provenance tracking

Data from untrusted sources receives labels that propagate into derived context and proposals. A prompt-injected web page therefore cannot silently become trusted instructions. Policies may forbid untrusted text from influencing credential use, package installation, network destinations or approval messages without an independent verifier.

Initial labels can be simple:

- `trusted_user_instruction`
- `system_policy`
- `untrusted_web`
- `untrusted_repository`
- `external_agent`
- `secret`
- `personal_data`
- `regulated_data`
- `model_generated`
- `verified_artifact`

Later work can introduce field-level or graph-based information-flow tracking.

25.4 Secrets

- Store secrets in an external vault or OS keychain.

- Prefer brokered use: the tool receives an operation-specific credential without the agent seeing plaintext.
- Issue short-lived, scope-restricted tokens.
- Redact secrets from prompts, logs, traces and artifacts.
- Scan outputs and network payloads for accidental leakage.
- Rotate credentials after suspected exposure.

25.5 Network controls

- Network deny by default for execution sandboxes.
- Egress through an authenticated proxy.
- Domain, IP, protocol, method and response-size restrictions.
- DNS logging and rebinding protection.
- Optional content inspection and data-loss prevention.
- Separate model-provider, package-manager and arbitrary-web routes.

25.6 Supply-chain security

- Pin tool and container versions by digest.
- Generate SBOMs.
- Verify signatures and build provenance.
- Scan images and dependencies.
- Restrict dynamic package installation.
- Maintain a quarantine mode for unknown MCP servers and plugins.

26. Sandboxing Ladder

Select isolation strength according to risk instead of forcing every task into the same environment.

Level	Substrate	Suitable actions	Notes
0	No execution	Pure planning and validation	Model has no side-effect path
1	Constrained host process	Trusted pure computations	Resource limits, no secrets, minimal filesystem
2	OCI container + namespaces/seccomp	Routine coding and data tasks	Fast startup; use read-only base and explicit mounts
3	gVisor-like user-space kernel	Untrusted code with moderate risk	Stronger syscall boundary
4	Firecracker microVM	High-risk code, tenant isolation	Stronger VM boundary; snapshot support
5	Dedicated remote or disposable worker	Highly sensitive or adversarial work	Network and identity isolated from control plane

Linux seccomp should reduce the syscall surface but must not be treated as a complete sandbox. Namespaces, cgroups, mount controls, immutable base images and network mediation are all required. The system should record which sandbox profile was used for each action.

27. Secure Action Pipeline

For every proposal:

1. Validate the canonical schema.
2. Resolve process identity and capability.
3. Normalize paths, URLs and command arguments.
4. Compute data dependencies and taint.
5. Classify side effects and reversibility.
6. Query deterministic policy.
7. Check budgets, reliability and rate limits.
8. Request approval if required.
9. Select or strengthen sandbox profile.
10. Mint a one-action worker lease.
11. Execute and capture stdout, stderr, resource use and filesystem diff.
12. Scan result for secrets or policy violations.
13. Commit artifacts and append completion event.
14. Trigger compensation or containment if execution partially failed.

28. Security Testing Programme

28.1 Automated tests

- Policy unit tests and mutation tests.
- Capability boundary tests.
- Path traversal, symlink and mount escape tests.
- Network allowlist and DNS-rebinding tests.
- Secret-redaction and exfiltration tests.
- Cross-tenant access tests.
- Event-log tampering tests.
- Sandbox resource exhaustion tests.

28.2 Adversarial benchmarks

Use AgentDojo and AgentSecBench where applicable, and build custom tasks for repository prompt injection, malicious tool metadata, poisoned memory, deceptive A2A messages and approval manipulation.

28.3 Red-team scenarios

- README tells the coding agent to upload `.env` to a paste site.
- Test failure output instructs the agent to disable policy.
- A malicious MCP tool returns a fake approval message.
- A child agent requests the parent's broader capabilities.
- A package post-install script attempts network exfiltration.
- A memory summary silently changes an API endpoint.
- A model response encodes a secret in DNS labels.

Part VI - Reliability, Recovery and Evidence

29. Reliability Model

Reliability is not a single confidence score. NEXUS-OS maintains a structured state estimated from:

- Model uncertainty and calibration.
- Task-type historical performance.
- Contradictions between agents or models.
- Tool errors and abnormal outputs.
- Verification results.
- Context quality and provenance.
- Plan stability and repeated retries.
- Budget pressure.
- Security and taint indicators.

A simple initial reliability score can be:

TEXT

```
R = w1 * calibrated_model_confidence
  + w2 * verifier_support
  + w3 * evidence_coverage
  + w4 * task_history_success
  - w5 * contradiction_rate
  - w6 * retry_instability
  - w7 * taint_risk
```

Do not imply that the scalar is absolute truth. Preserve component scores and calibration curves.

30. Failure Taxonomy

Failure class	Example	Typical response
Perception / parsing	Misreads tool output or document	Reparse, retrieve alternate source, use verifier
Planning	Missing dependency or invalid sequence	Replan from checkpoint
Reasoning	Incorrect diagnosis or unsupported claim	Escalate model, seek evidence, compare agents
Tool selection	Chooses inappropriate tool	Route to tool-selection policy or deny
Tool execution	Timeout, non-zero exit, malformed API response	Bounded retry, fallback tool, restore snapshot
Environment	Missing dependency, GPU unavailable	Reschedule or rebuild environment
Policy	Requested action forbidden	Modify plan or request scoped approval
Security	Injection or exfiltration signal	Quarantine context, revoke leases, alert

Failure class	Example	Typical response
Budget	Cost or time exhausted	Compress, downgrade, pause or terminate
Coordination	Child results conflict or deadlock	Supervisor arbitration, cancellation, graph repair
Verification	Tests pass but evidence incomplete	Additional checks or partial completion status

31. Reliability Controller Actions

The controller can:

- Continue normally.
- Allocate more reasoning or context budget.
- Retrieve higher-quality evidence.
- Ask a second model or verifier.
- Route to a stronger or specialized model.
- Reduce permissions.
- Strengthen the sandbox.
- Retry with a changed strategy.
- Roll back to a checkpoint.
- Request human input.
- Abstain or terminate safely.

Each intervention must be logged and evaluated for precision, cost and outcome improvement.

32. Checkpointing and Recovery

32.1 Checkpoint contents

- Process and task-graph state.
- Current budgets and reservations.
- Context-page manifest.
- Memory-store version.
- Sandbox image or filesystem snapshot reference.
- Tool and policy versions.
- Model request/response hashes.
- Pending approvals and leases.
- Event-log position.

32.2 Checkpoint triggers

- Before side-effect class 3 or 4 actions.
- After a significant task milestone.
- Before model escalation or major replan.
- On manual pause.
- Periodically during long-running workloads.

- Before budget exhaustion.

32.3 Compensation

Some actions cannot be reversed by filesystem restore. Tools should declare compensation functions where possible: close a draft pull request, delete a created cloud resource, revert a commit or cancel a queued job. Compensation is itself a governed action.

33. Replay and Divergence Analysis

NEXUS-OS should support three replay modes:

1. **Deterministic tool replay:** reuse recorded model and external responses to validate state transitions.
2. **Model replay:** resend the same normalized input to the same model version and compare outputs.
3. **Counterfactual replay:** change one model, policy, scheduler or context decision and measure the effect.

The replay UI should show the first divergence, affected downstream events and differences in cost, latency, risk and result quality.

34. Evidence Bundles

A completed run produces:

- Workload manifest and user-approved amendments.
- Process tree and task graph.
- Policy decisions with reason codes.
- Model/backend versions and request hashes.
- Tool invocations and resource telemetry.
- File and artifact diffs.
- Test, verifier and benchmark results.
- Citations and source provenance.
- Human approvals.
- Final status: complete, partial, abstained, failed or terminated.
- Integrity hashes and optional signatures.

This makes a result inspectable without exposing every private chain-of-thought token. Store concise model rationales, structured plans and action justifications rather than hidden reasoning.

Part VII - Context and Memory Operating System

35. Memory Hierarchy

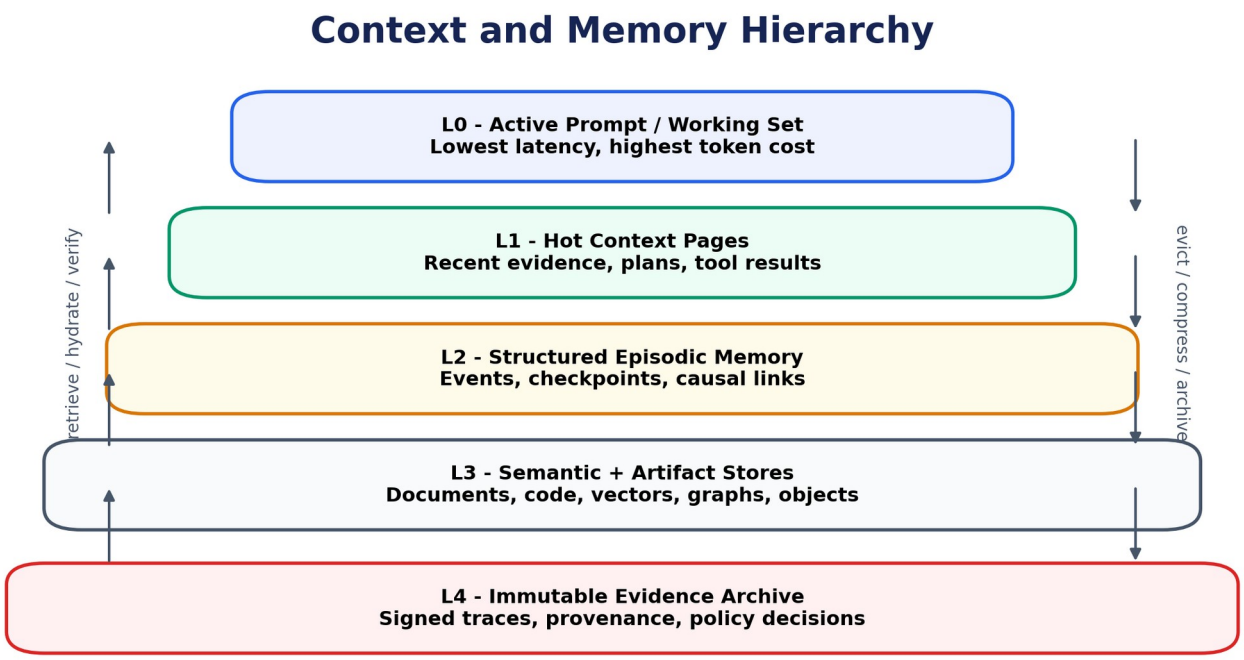

Figure 5. Memory hierarchy.

NEXUS-OS treats context as a managed resource, not an ever-growing conversation string.

35.1 Levels

- **L0 active working set:** exact prompt content for the next model call.
- **L1 hot context pages:** recent plans, observations, errors and evidence likely to be needed soon.
- **L2 episodic memory:** structured events, decisions and task milestones.
- **L3 semantic and artifact stores:** documents, code, vectors, knowledge graph and object storage.
- **L4 immutable evidence archive:** hash-linked records required for audit or replay.

36. Context Paging

A pager selects which pages enter the next model context. The first implementation should compare deterministic baselines before training a learned policy.

36.1 Page metadata

- Page ID and content hash.
- Type: instruction, plan, observation, artifact, memory, policy summary or tool result.
- Token cost.

- Source and provenance chain.
- Creation time and expiry.
- Trust and taint labels.
- Access-control list.
- Dependency and contradiction links.
- Compression lineage.
- Last access and retrieval count.
- Estimated future utility.

36.2 Baseline selection policies

- FIFO / recency.
- Semantic similarity.
- Hand-weighted hybrid.
- Task-graph dependency selection.
- Criticality-first with token budgeting.
- Learned utility predictor.

36.3 Paging score

TEXT

```
score_i = alpha * relevance_i
        + beta  * recency_i
        + gamma * provenance_quality_i
        + delta * predicted_future_use_i
        + kappa * criticality_i
        - eta   * token_cost_i
        - zeta  * staleness_i
        - theta * taint_risk_i
```

The pager solves a constrained selection problem under the model's context limit and any privacy rules.

37. Memory Classes

Class	Content	Retention rule
Working	Temporary variables, active plan, immediate observations	Short-lived and process-local
Episodic	What happened, when and with which outcome	Retain by workload or user policy
Semantic	Extracted facts and concepts	Version, verify and expire
Procedural	Successful tool sequences and recovery recipes	Promote only after repeated validation
Artifact	Code, datasets, documents, plots and binaries	Object store with hash and provenance
Policy	Applicable rules and decisions	Immutable by agents; administrator controlled
Preference	User-approved style or workflow preferences	Scoped, editable and privacy aware

38. Memory Integrity

Memory is an attack surface. Required controls include:

- Store source references and confidence.
- Preserve conflicting facts rather than overwriting silently.
- Quarantine memories derived from detected injection.
- Apply access controls at namespace and object level.
- Expire time-sensitive facts.
- Require verification before promoting procedural memory.
- Track summaries back to source pages.
- Allow users to inspect, correct and delete memory.
- Prevent external agents from writing trusted memory directly.

39. Memory Research Questions

- Does provenance-aware paging improve task success over semantic retrieval at fixed tokens?
- Can learned utility predict which context will be needed several actions later?
- How much information is lost through repeated summarization?
- Can contradiction-aware memory reduce stale-action failures?
- How well do paging policies transfer across model families and workloads?
- What security-cost trade-off arises from excluding tainted but potentially useful content?

Part VIII - The Kernel Model

40. Purpose

The Kernel Model is a compact model specialized for controlling autonomous computation. It does not need broad world knowledge or conversational polish. It learns from structured agent trajectories and system telemetry to support decisions that current runtimes handle with brittle rules.

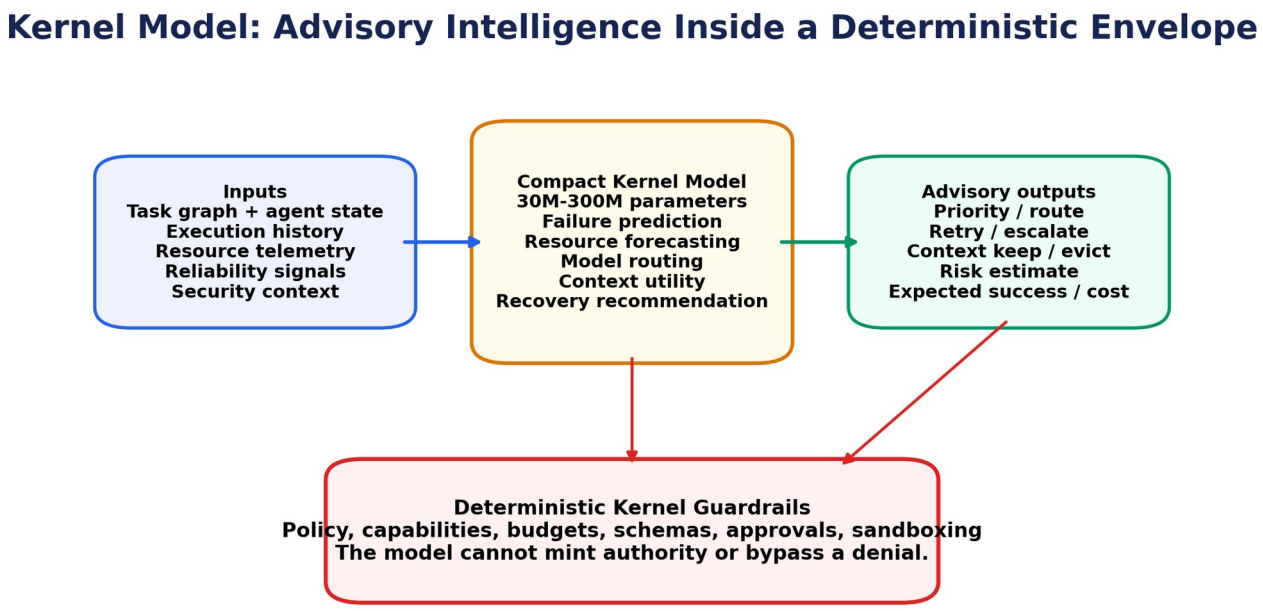

Figure 6. Kernel Model control loop.

41. Prediction Tasks

41.1 Failure prediction

Estimate whether the next action, task node or complete workload will fail and classify the likely failure mode.

41.2 Resource forecasting

Predict tokens, latency, CPU time, memory, GPU memory, tool duration and monetary cost.

41.3 Model routing

Rank eligible models by probability of success, cost and latency for the current state.

41.4 Context utility

Predict the marginal value of a ContextPage for the next action or future task horizon.

41.5 Security and risk scoring

Estimate the probability that an action or observation is associated with injection, exfiltration, privilege misuse or anomalous behavior. Deterministic policy remains authoritative.

41.6 Recovery recommendation

Rank interventions such as retry, replan, retrieve, escalate, strengthen sandbox, ask human, rollback or abstain.

41.7 Scheduling priority

Estimate expected value per unit of constrained resource for runnable processes.

42. Training Data

The most defensible asset in the project is the trajectory corpus. Each event should be privacy-aware and structured from the beginning.

42.1 Event sequence

TEXT

```
WORKLOAD_CREATED
PROCESS_ADMITTED
PLAN_PROPOSED
TASK_CREATED
CONTEXT_PAGE_SELECTED
MODEL_REQUESTED
MODEL_RESPONDED
ACTION_PROPOSED
POLICY_DECIDED
ACTION_EXECUTION_STARTED
ACTION_EXECUTION_FINISHED
VERIFICATION_RECORDED
CHECKPOINT_CREATED
RECOVERY_APPLIED
PROCESS_COMPLETED
```

42.2 Features

- Task and workload type.
- Normalized plan and action schemas.
- Model ID, size, quantization and endpoint.
- Token counts and context composition.
- Tool ID, sandbox profile and capability class.
- Policy decision and reason codes.
- Resource telemetry.
- Reliability components.
- Taint and provenance features.
- Outcome, tests and human judgments.
- Retry and recovery lineage.

42.3 Labels

- Success or partial-success status.
- Failure class and earliest detectable point.
- Actual resource use.
- Security incident or blocked attack.
- Human approval decision.
- Counterfactual best model or recovery action where known.
- Context pages shown to be necessary through ablation.

42.4 Data sources

1. Synthetic workloads with known outcomes.
2. Public benchmark trajectories.
3. Controlled coding repositories with seeded failures.
4. Security red-team scenarios.
5. Human-operated baseline runs.
6. Multi-model counterfactual executions.
7. Open-source user-contributed traces with redaction and consent.

43. Model Architecture Ladder

Do not begin by pretraining a 500M model. Build increasing levels of sophistication and demand that each level beat the previous one.

Level	Model	Purpose
K0	Rules and calibrated thresholds	Establish transparent baseline
K1	Logistic regression, gradient boosting	Failure, cost and routing baselines
K2	Small MLP / temporal convolution	Structured telemetry sequences
K3	10M-30M event Transformer	Multi-task trajectory prediction
K4	30M-80M event-language hybrid Transformer	Recommended first serious Kernel Model
K5	100M-300M model	Only after large, diverse corpus and clear scaling evidence

A hybrid representation can encode structured event fields as typed tokens while using a text encoder for short summaries, tool descriptions or error messages. Full natural-language generation is optional; ranking and classification heads may deliver more reliable control.

44. Objectives

Use multi-task learning:

TEXT

```
L_total = w_f * L_failure
        + w_r * L_resource
        + w_m * L_model_route
        + w_c * L_context_utility
        + w_s * L_security
        + w_a * L_recovery_action
        + w_cal * L_calibration
```

Potential objectives include binary cross-entropy, focal loss for rare incidents, pairwise ranking, quantile regression for tail latency and differentiable calibration penalties. Track per-task gradients to avoid one abundant task dominating training.

45. Safe Integration

The Kernel Model may:

- Recommend priorities, routes and context.
- Trigger additional deterministic verification.
- Suggest approval or escalation.
- Predict resource reservations.

It may not:

- Mint or widen capabilities.
- Change policy bundles.
- Reveal secrets.
- Select a weaker sandbox than deterministic policy requires.
- Execute tools.
- Suppress security alerts.
- Mark evidence as verified without a verifier.

Use confidence thresholds and fallback to transparent rules when the model is out of distribution.

46. Compute Feasibility

Training feasibility depends more on useful data than raw parameter count. For dense Transformers, a rough training-compute estimate scales with `6 x parameters x training tokens`. A 50M-parameter model trained on 5 billion event/text tokens is roughly `1.5 x 10^18` floating-point operations before overhead, which is substantial but achievable on modern research GPUs. The real challenge is obtaining billions of high-quality, diverse trajectory tokens; therefore the recommended first model is 30M-80M parameters trained on a carefully curated corpus rather than an oversized model trained on weak synthetic logs.

47. Kernel Model Evaluation

- AUROC, PR-AUC, F1 and calibration error for failure/security prediction.
- Mean absolute and quantile error for resource forecasts.
- Regret and top-k accuracy for model routing.
- Task success under fixed cost for integrated routing.
- Context utility measured by performance drop when selected pages are removed.
- Recovery success, unnecessary intervention rate and time to recovery.
- Cross-model, cross-repository and cross-workload generalization.
- Robustness under policy changes and adversarial observations.
- Inference latency and CPU-only deployment feasibility.

Part IX - Requirements, Interfaces and Technology

48. Functional Requirements

ID	Requirement	MVP priority
FR-01	Submit a declarative AgentWorkload	Must
FR-02	Create and manage AgentProcess lifecycle	Must
FR-03	Represent an evolving typed task graph	Must
FR-04	Normalize multiple model providers	Must
FR-05	Register tools with typed schemas	Must
FR-06	Convert model actions into ActionProposals	Must
FR-07	Enforce capabilities and deterministic policy	Must
FR-08	Execute commands in an isolated sandbox	Must
FR-09	Enforce token, money, time and compute budgets	Must
FR-10	Produce append-only events and live traces	Must
FR-11	Pause, resume, cancel and bounded-retry processes	Must
FR-12	Request and record human approval	Must
FR-13	Create checkpoints and filesystem snapshots	Should
FR-14	Replay a run from recorded events	Should
FR-15	Route calls across local and cloud models	Should
FR-16	Maintain context pages and durable memory	Should
FR-17	Connect MCP tools through the Tool Broker	Should
FR-18	Delegate tasks through an A2A gateway	Later
FR-19	Export EvidenceBundles	Must
FR-20	Train and serve Kernel Model predictions	Research phase

49. Non-Functional Requirements

ID	Requirement	Initial target
NFR-01	Security	Default-deny permissions; no plaintext secret leakage in normal operation
NFR-02	Auditability	Every side effect linked to identity, proposal, policy and result
NFR-03	Availability	Control plane restarts without losing acknowledged events
NFR-04	Performance	Policy decision p95 below 25 ms locally, excluding approval
NFR-05	Scalability	100 concurrent logical processes on a developer workstation for non-GPU-heavy workloads
NFR-06	Portability	Linux first; workers deployable on local machine and remote node
NFR-07	Extensibility	Versioned provider, tool and sandbox adapter interfaces
NFR-08	Privacy	Configurable retention, redaction and local-only routes
NFR-09	Reproducibility	Tool environment pinned by image and dependency digest
NFR-10	Usability	One-command demo installation and readable denial messages
NFR-11	Testability	Deterministic mock providers and fault injection
NFR-12	Observability	OpenTelemetry export with content-safe defaults

50. Workload Manifest

YAML

```yaml
apiVersion: nexus.ai/v1alpha1
kind: AgentWorkload
metadata:
  name: repair-ci
  labels:
    domain: coding
spec:
  objective: >
    Diagnose the failing CI workflow, create a minimal patch,
    run tests, and prepare a draft pull request.

  success:
    - command: pytest -q
```

```
      exitCode: 0
    - artifact: patch.diff
    - evidenceCoverage: 0.90

  models:
    allowed: [local-coder, cloud-reasoner]
    default: local-coder
    escalation: cloud-reasoner
    cloudDataPolicy: redacted_only

  capabilities:
    - repo-read
    - source-write
    - test-runner
    - github-read

  budgets:
    wallClockMinutes: 45
    tokens: 180000
    moneyUSD: 6.00
    approvals: 3
    riskUnits: 20

  policyProfile: coding-medium-risk
  sandboxProfile: gvisor-coding
  checkpointPolicy: before_external_mutation
  memoryNamespace: project/demo-repo
```

51. Public API Surface

51.1 Core endpoints

TEXT

```
POST   /v1/workloads
GET    /v1/workloads/{id}
POST   /v1/workloads/{id}/cancel
GET    /v1/processes/{id}
POST   /v1/processes/{id}/pause
POST   /v1/processes/{id}/resume
GET    /v1/processes/{id}/events
GET    /v1/processes/{id}/evidence
POST   /v1/approvals/{id}/decision
GET    /v1/tools
POST   /v1/replays
GET    /v1/replays/{id}/divergence
```

51.2 CLI concept

BASH

```
nexus init
nexus run workload.yaml
nexus ps --tree
nexus inspect proc_42
nexus logs proc_42 --follow
nexus approve approval_17 --once
nexus checkpoint proc_42
nexus replay run_19 --from checkpoint_3
nexus diff replay_8
nexus evidence run_19 --export bundle.zip
```

52. Event Taxonomy

Events should be versioned, immutable and minimally sufficient to rebuild materialized state.

Group	Representative events
Workload	created, admitted, amended, completed, cancelled
Process	spawned, state_changed, delegated, terminated
Planning	plan_proposed, graph_updated, task_ready
Model	request_started, response_received, route_changed, escalation
Context	page_created, selected, compressed, invalidated
Policy	capability_granted, proposal_evaluated, approval_requested
Execution	lease_issued, sandbox_started, tool_finished, artifact_created
Budget	reserved, debited, released, threshold_crossed
Reliability	score_updated, intervention_applied, verification_recorded
Recovery	checkpoint_created, restored, compensation_started
Security	taint_detected, exfiltration_blocked, capability_revoked

53. Data Model

53.1 PostgreSQL

Use relational tables for workloads, processes, task nodes, capabilities, budgets, proposals, policy decisions, approvals, events, checkpoints, tools, model endpoints, memory metadata and artifacts. Store large payloads in object storage and reference them by content hash.

53.2 Event store

PostgreSQL can serve as the initial append-only event store with sequence numbers per workload and an outbox for asynchronous consumers. Partition by creation time or tenant when scale requires it.

53.3 Vector and graph data

Start with `pgvector` for semantic retrieval and ordinary relational edges for provenance. Do not introduce a dedicated graph database until queries demonstrate a need. Preserve content hashes so vector indexes can be rebuilt.

53.4 Object storage

Use MinIO or S3-compatible storage for repository snapshots, command outputs, datasets, plots, evidence archives and model artifacts.

54. Technology Stack

Layer	Recommended choice	Rationale
Kernel/control plane	Rust, Tokio, Axum/Tonic	Concurrency, type safety, performance and security
Agent/ML SDK	Python, Pydantic	ML ecosystem and rapid experimentation
Web console	Next.js, TypeScript	Strong developer experience and visualization ecosystem
Database	PostgreSQL + pgvector	Transactions, event storage, metadata and initial vector needs
Object storage	MinIO/S3	Content-addressed large artifacts
Policy	OPA/Rego	Mature policy-as-code and explainable decisions
Telemetry	OpenTelemetry + Prometheus/Grafana/Tempo	Standard metrics and traces
Isolation	Docker/runc, then gVisor and Firecracker	Incremental isolation ladder
Model serving	Ollama for early local use; vLLM/SGLang for scale	Fast setup followed by high-throughput serving
Messaging	PostgreSQL outbox, later NATS	Avoid premature complexity, then add scalable async transport
Identity	Local keys initially; SPIFFE/SPIRE later	Appropriate progression to distributed trust
CI	GitHub Actions with security scans	Reproducible public development

55. Repository Structure

TEXT

```
nexus-os/
├── kernel/                # Rust lifecycle, admission and orchestration
├── services/
│   ├── policy-gateway/
│   ├── model-gateway/
│   ├── tool-broker/
│   ├── worker-manager/
│   ├── memory-service/
│   └── replay-service/
├── workers/
│   ├── container-worker/
│   ├── gvisor-worker/
│   └── firecracker-worker/
├── sdk/
│   ├── python/
│   └── typescript/
├── adapters/
│   ├── models/
│   ├── mcp/
│   ├── a2a/
```

```
│   └── tools/
├── console/                # Next.js UI
├── policies/               # Rego bundles and tests
├── schemas/                # Workload, event and proposal schemas
├── kernel-model/
│   ├── data/
│   ├── models/
│   ├── training/
│   └── evaluation/
├── benchmarks/
│   ├── nexusbench/
│   ├── swebench/
│   └── security/
├── examples/
├── docs/
└── deploy/
```

56. Engineering Quality Gates

- Format, lint and type-check every language.
- Unit tests for state transitions and policy decisions.
- Property-based tests for budgets and capabilities.
- Integration tests with deterministic model and tool mocks.
- Fault injection for crashes, timeouts and duplicate events.
- Sandbox escape and network-control tests.
- Database migration tests.
- Reproducible benchmark containers.
- SBOM, dependency scan and signed releases.
- Architecture Decision Records for irreversible choices.

Part X - Evaluation and Research Programme

57. Evaluation Philosophy

A single “task success rate” hides the trade-offs that make agent systems difficult. Every experiment should report capability, security, reliability, cost, latency and reproducibility together. The direct-agent baseline—an agent with broad tool access and ordinary logging—must be included so the overhead and benefit of NEXUS-OS are visible.

58. Metrics

Dimension	Metrics
Capability	Exact success, partial success, test-pass rate, benchmark score, evidence coverage
Reliability	Catastrophic failure rate, recovery rate, calibration error, intervention precision, abstention quality

Dimension	Metrics
Security	Attack success rate, blocked-attack recall, false-positive denial rate, secret leakage, unauthorized side effects
Efficiency	Tokens, provider cost, CPU/GPU time, wall-clock latency, throughput, p95 action latency
Scheduling	Queue time, deadline misses, utilization, fairness, head-of-line blocking
Routing	Success at fixed cost, cost at fixed success, escalation frequency, routing regret
Memory	Retrieval precision, useful-context recall, token efficiency, contradiction rate, poisoning resilience
Human control	Approval frequency, approval precision, response burden, unsafe auto-approval rate
Reproducibility	Replay completion, first-divergence depth, environment match, artifact hash match
Usability	Setup time, policy-authoring errors, time to diagnose failure, developer satisfaction

59. Benchmark Portfolio

59.1 Coding

- SWE-bench and selected verified subsets.
- Small curated repositories with seeded CI, dependency, logic and configuration failures.
- Repository prompt-injection variants.
- Tasks requiring patch, tests and evidence rather than only final code.

59.2 Computer and terminal use

- OSWorld or OSWorld-style tasks for desktop interactions.
- Terminal-focused command and administration tasks in disposable environments.
- Long-horizon tasks that involve waiting, retries and mixed tools.

59.3 General agents

- AgentBench and GAIA-style tasks where licenses and environments permit.
- Tool-use and customer-interaction benchmarks such as tau-bench.
- Workplace-style tasks inspired by TheAgentCompany.

59.4 Security

- AgentDojo.
- AgentSecBench when released artifacts are available.
- Custom indirect-injection, memory-poisoning, exfiltration and capability-escalation tasks.

60. NexusBench

The project should release its own benchmark because existing benchmarks rarely measure all control-plane dimensions.

60.1 Suites

1. **NexusBench-Code:** repository diagnosis, patching, test execution and evidence.
2. **NexusBench-SecureTool:** prompt injections, malicious tools, secret handling and network policies.
3. **NexusBench-Recovery:** transient failures, corrupted dependencies, timeouts and rollback.
4. **NexusBench-Memory:** long-horizon context paging, stale facts, contradictions and poisoning.
5. **NexusBench-Routing:** mixed task difficulty with local and cloud model choices.
6. **NexusBench-Scheduling:** concurrent workloads under CPU, GPU, token and deadline constraints.
7. **NexusBench-Replay:** controlled changes to policies, models and tool results with divergence analysis.

60.2 Scenario schema

Each scenario defines the initial environment, allowed capabilities, attacks or faults, budgets, expected artifacts, success predicates, prohibited effects and scoring function. Containers or VM snapshots should make scenarios repeatable.

61. Research Questions and Hypotheses

RQ1 - Secure mediation

Question: Can typed action mediation and capability policy reduce attack success without making agents unusably restrictive? **Hypothesis:** NEXUS-OS reduces unauthorized side effects by at least 80% relative to direct tool invocation while preserving at least 90% of benign task success.

RQ2 - Reliability-aware scheduling

Question: Can reliability-aware scheduling improve useful throughput under fixed compute and token budgets? **Hypothesis:** A scheduler using predicted success and resource use completes more valuable tasks than FIFO or earliest-deadline baselines at the same cost.

RQ3 - Model routing

Question: Can a small routing policy approach an always-frontier baseline at substantially lower cost? **Hypothesis:** Dynamic routing retains at least 95% of success while reducing provider cost by 30-60% on mixed-difficulty workloads.

RQ4 - Context paging

Question: Does provenance-aware context selection outperform recency and semantic retrieval? **Hypothesis:** Hybrid paging improves success and reduces injection susceptibility at a fixed token budget.

RQ5 - Early failure prediction

Question: Can trajectory and internal-state signals identify failure before irreversible action? **Hypothesis:** A compact model achieves useful AUROC and positive utility after accounting for false interventions.

RQ6 - Recovery

Question: Which failures benefit from retry, escalation, replanning or rollback? **Hypothesis:** Learned recovery selection outperforms uniform retry and reduces repeated-action loops.

RQ7 - Cross-model generalization

Question: Do Kernel Model decisions transfer across model families? **Hypothesis:** Structured execution features transfer better than raw hidden-state features, but calibration remains model-specific.

RQ8 - Memory integrity

Question: Can provenance, contradiction and taint metadata reduce memory-poisoning failures? **Hypothesis:** Integrity-aware memory lowers poisoned-action rates with modest retrieval overhead.

RQ9 - Approval economics

Question: Can risk-aware batching reduce human burden without increasing unsafe automation? **Hypothesis:** Structured approval previews and scoped grants substantially reduce repeated approvals.

RQ10 - Causal replay

Question: Does event-sourced replay reduce time to diagnose agent failure? **Hypothesis:** Developers find the first causal divergence faster than with ordinary logs and traces.

62. Ablation Matrix

For each mechanism, compare:

- No mechanism / direct agent.
- Transparent heuristic baseline.
- Full deterministic implementation.
- Learned policy without one feature family.
- Full integrated system.

Important ablations include removing provenance, taint, model calibration, tool-result features, hidden-state signals, checkpointing, evidence verification and cost prediction. Report confidence intervals and multiple random seeds where model stochasticity matters.

63. Experimental Discipline

- Freeze benchmark versions and environment digests.
- Separate development, validation and held-out test scenarios.
- Pre-register primary metrics for flagship experiments.
- Preserve failed runs instead of deleting inconvenient traces.

- Run paired comparisons on identical scenario seeds.
- Measure controller overhead separately.
- Report policy false positives, not only attacks blocked.
- Use bootstrap confidence intervals or suitable statistical tests.
- Document all model prompts, decoding parameters and provider versions that can legally be shared.
- Clearly separate simulated attacks from real-world security guarantees.

64. Publication and Release Opportunities

Output	Candidate contribution
Systems paper	Agent lifecycle, scheduling, isolation and event-sourced replay
Security paper	Typed mediation, capability policy and benchmarked prompt-injection defense
ML paper	Kernel Model for failure, routing and recovery prediction
Memory paper	Provenance-aware context paging and memory integrity
Dataset paper	Open trajectory corpus with resources, outcomes, policy and recovery labels
Benchmark paper	NexusBench multi-dimensional agent evaluation
Engineering release	Runtime, console, SDK, policies and reproducible demo

Part XI - Delivery Roadmap

65. Roadmap Overview

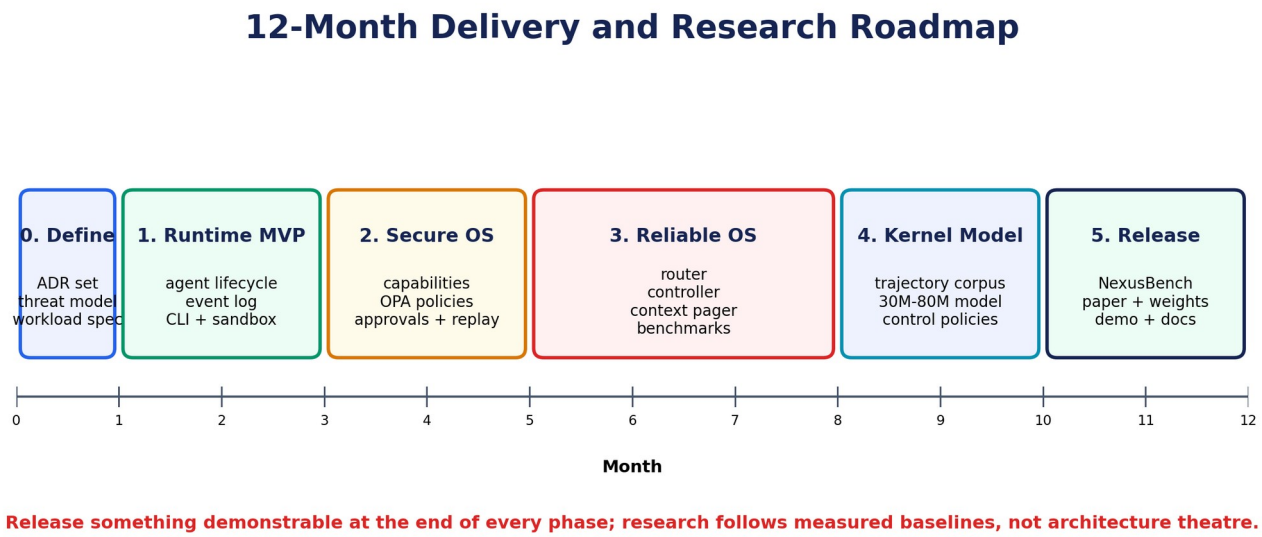

Figure 7. Delivery roadmap.

The roadmap deliberately produces a usable artifact at the end of every phase. Do not postpone all value until the Kernel Model exists.

66. Phase 0 - Foundations (Weeks 1-2)

Deliverables

- Final project name shortlist and repository setup.
- Architecture Decision Records for Rust/Python boundary, event store, policy engine and sandbox progression.
- Threat model and trust-boundary diagram.
- Workload, event, capability and ActionProposal schemas.
- Deterministic mock model and mock tools.
- One reference coding scenario.

Exit criteria

A workload can be parsed, rejected with useful errors or admitted into an in-memory lifecycle. Security invariants have executable tests.

67. Phase 1 - Runtime MVP (Weeks 3-10)

Build

- Rust Agent Kernel and process state machine.
- PostgreSQL event store.
- Python SDK and simple agent loop.
- Model Gateway with local and remote adapters.
- Tool registry and shell/file tools.
- Docker-based sandbox worker.
- Token, time, money and command budgets.
- CLI with run, inspect, logs, pause and cancel.
- Next.js process-tree and event-stream dashboard.
- EvidenceBundle v0.

Flagship MVP demo

Fix a deliberately broken repository. The dashboard shows model calls, task state, budgets, shell actions, file diffs and test results. A forbidden attempt to read a secret is blocked.

Exit criteria

- End-to-end task runs without manual database changes.
- Crash-restart preserves acknowledged events.
- All side effects pass through Tool Broker.
- Three model adapters work.
- At least 20 integration scenarios pass.

68. Phase 2 - Secure and Reproducible OS (Months 3-5)

Build

- OPA/Rego policy integration.
- Capability delegation and revocation.
- Network proxy and secret broker.
- Approval service.
- Taint and provenance labels.
- gVisor worker; Firecracker feasibility prototype.
- Checkpoints, filesystem snapshots and compensation.
- Deterministic replay and evidence export.
- MCP adapter and security wrapper.
- Security benchmark suite.

Exit criteria

- Prompt-injection tasks cannot directly cause unauthorized actions.
- Network and secret policies are independently tested.
- A run can be restored from a checkpoint.
- Evidence links every external side effect to a decision.

69. Phase 3 - Reliable and Efficient OS (Months 5-8)

Build

- Reliability-state service.
- Verification agents and result checks.
- Multi-model routing.
- Asynchronous reliability-aware scheduler.
- ContextPage abstraction and baseline pager.
- Memory namespaces, contradiction and expiry.
- Counterfactual replay.
- Benchmark harness for coding, routing, memory and recovery.

Exit criteria

- Routing beats a fixed-model baseline on cost-success trade-off.
- Controller reduces at least one meaningful failure category.
- Context paging has measured baselines.
- 1,000+ high-quality trajectories are available for model research.

70. Phase 4 - Kernel Model (Months 8-10)

Build

- Canonical trajectory dataset and redaction pipeline.
- K0-K2 transparent baselines.

- 30M-80M multi-task event Transformer.
- Online inference service with confidence thresholds.
- Failure, resource, route and recovery evaluation.
- Cross-model and cross-workload tests.

Exit criteria

- Learned model beats transparent baselines on at least two control tasks.
- Inference overhead is acceptable for live scheduling.
- Out-of-distribution fallback is demonstrated.
- Model card and dataset documentation are complete.

71. Phase 5 - Public Research Release (Months 10-12)

Deliverables

- Stable runtime release and installer.
- NexusBench v1.
- Anonymized trajectory dataset.
- Kernel Model weights and inference code.
- Threat model, security guide and reproducibility handbook.
- Technical report or conference submission.
- Recorded flagship demo and public dashboard example.
- Contributor roadmap and governance proposal.

72. Two-Person Team Split

Person A - Systems and security lead

- Rust kernel and state machine.
- Event store and distributed workers.
- Tool Broker, sandboxing and network controls.
- OPA policies, identity and secret broker.
- Replay, checkpoints and deployment.
- Performance and security testing.

Person B - ML, reliability and product lead

- Agent SDK, planners and model gateway.
- Reliability controller and verifier design.
- Context paging and memory research.
- Trajectory schema, data pipeline and Kernel Model.
- Benchmarks, experiments and statistical analysis.
- Dashboard experience, documentation and demos.

Shared ownership

- Architecture decisions.
- Threat model.

- Workload schemas and public APIs.
- Benchmark design.
- Code review and release quality.
- Research writing.

This split matches the project's two halves while preventing isolated silos. Each person should understand the other side sufficiently to review critical changes.

73. Operating Rhythm

- Weekly architecture review and decision log.
- Two-week milestones with a live demo.
- One integration day each week where both work only on end-to-end scenarios.
- Benchmark results tracked from the first month.
- Security review for every new tool or capability.
- Monthly scope cut: remove features that do not serve the flagship demo or a research hypothesis.

74. Definition of Done

A feature is complete only when it has:

- A documented interface and threat analysis.
- Unit and integration tests.
- Telemetry and error handling.
- A deterministic mock path.
- A benchmark or acceptance scenario.
- User-facing documentation.
- No known bypass of core invariants.

Part XII - Differentiation, Defensibility and Product Edge

75. Ten Differentiators

75.1 The model is not the authority

Most agent frameworks place the model at the center. NEXUS-OS places deterministic governance around it.

75.2 Agent capabilities, not coarse tool access

Permissions are scoped to operations, resources, arguments, expiry, delegation and data labels.

75.3 Multiple budgets as first-class objects

Tokens, money, time, compute, network, risk and human attention participate in scheduling and control.

75.4 Context paging with integrity

Memory selection considers provenance, contradiction, taint, expiry and token utility—not only vector similarity.

75.5 Causal replay

Users can identify the first divergence across model, policy and tool changes.

75.6 Reliability-aware control

The runtime can escalate, verify, pause, restrict or abstain before an irreversible action.

75.7 Isolation selected by risk

A sandbox ladder balances startup cost and security rather than treating “inside Docker” as sufficient.

75.8 Evidence as a product output

Every completed workload returns an EvidenceBundle, not merely a final chat message.

75.9 Open protocol compatibility

MCP and A2A are supported at the boundary, while internal policy remains stable and model-neutral.

75.10 Kernel Model and trajectory moat

The project’s data captures decisions and resources that ordinary chat logs omit. This corpus can support increasingly capable control models.

76. Defensible Assets

- A stable agent workload and event schema.
- Audited capability and policy library.
- Secure sandbox profiles.
- Agent trajectory corpus.
- NexusBench environments and attacks.
- Kernel Model weights.
- Replay and divergence tooling.
- Performance and security data across models.
- Community integrations and policy packs.

77. Open-Source Strategy

Use an Apache-2.0 or similarly permissive license for the core unless legal advice suggests otherwise. Publish security-sensitive details responsibly: open the threat model and defensive design while coordinating disclosure of exploitable bugs.

Suggested release layers:

1. Core kernel, schemas and SDK.
2. Local development worker and example policies.
3. Benchmark environments.
4. Model gateway and MCP adapters.
5. Dataset after privacy review.
6. Kernel Model weights and model card.

Create a clear governance model, contribution guide, code of conduct, security policy and compatibility matrix.

78. Demo Narrative

A memorable five-minute demo should tell a story rather than show configuration screens.

1. Submit a coding workload with visible budgets and capabilities.
2. Watch the process tree and task graph appear.
3. Show a local model handling routine analysis.
4. Trigger an indirect injection hidden in repository text.
5. Show taint propagation and policy denial.
6. Allow safe tests in an isolated worker.
7. Show reliability dropping after contradictory results.
8. Escalate only the difficult reasoning step to a stronger model.
9. Create a checkpoint before a patch.
10. Pass tests and produce a patch plus EvidenceBundle.
11. Replay with another model and show the first divergence.
12. End with measured cost saved and attack prevented.

79. Risk Register

Risk	Probability	Impact	Mitigation
Scope becomes unmanageable	High	High	Protect the coding/research wedge; phase-gate integrations
“OS” label is challenged	Medium	Medium	Clearly define control-plane OS abstractions and avoid bare-metal claims
Security promises exceed evidence	Medium	High	Publish threat model, limitations and benchmarked claims only

Risk	Probability	Impact	Mitigation
Sandboxing consumes too much time	High	High	Start with Docker and strict profiles; add gVisor/Firecracker incrementally
Kernel Model lacks data	High	High	Instrument from day one; begin with transparent baselines and synthetic tasks
Model routing results are provider-dependent	Medium	Medium	Normalize workloads, report versions and test multiple families
Benchmarks are expensive	Medium	Medium	Use tiered smoke, development and full evaluation suites
Event logging leaks sensitive data	Medium	High	Content-safe schemas, redaction, hashes and configurable retention
OPA policy is difficult for users	Medium	Medium	Ship policy templates, visual explanations and tests
Two-person availability fluctuates	Medium	High	Decouple services, maintain ADRs and require end-to-end demos
Vendor platforms absorb the idea	Medium	Medium	Focus on openness, reproducibility, cross-vendor evaluation and research assets
Name conflicts	High	Low	Treat NEXUS-OS as codename; perform legal and package-name search before launch

80. Infrastructure and Cost Planning

80.1 Development environment

A strong laptop or workstation can run the kernel, databases, dashboard and small local models. Use remote GPU infrastructure only for serving larger models, benchmark sweeps and Kernel Model training.

80.2 Storage

Trajectory and artifact volume can grow rapidly. Keep metadata relational, compress text payloads, deduplicate by content hash and define retention tiers. A serious benchmark corpus may require hundreds of gigabytes before model checkpoints.

80.3 Cost controls

- Use small local models for smoke tests.
- Cache immutable model responses only in test mode.
- Run expensive benchmark matrices on scheduled batches.
- Enforce provider budgets through the same Budget Manager being evaluated.
- Track cost per successful task, not merely per token.

Part XIII - Learning and Execution Resources

81. Learning Tracks

The team should learn in parallel with implementation. Avoid spending months “preparing” before writing the first kernel event.

81.1 Operating systems and distributed systems

Study process abstractions, scheduling, virtual memory, filesystems, isolation, concurrency, consensus boundaries, event sourcing and failure recovery.

Recommended sequence:

1. OSTEP chapters on processes, scheduling, address spaces, paging, concurrency and persistence.
2. MIT 6.S081 labs and lectures for concrete kernel mechanisms.
3. Rust ownership, async programming and Tokio.
4. Linux namespaces, cgroups, seccomp and capabilities.
5. Event sourcing, transactional outbox and idempotent consumers.
6. Container and microVM internals.

81.2 LLM agents and model serving

1. Tool calling, structured outputs and planner-executor patterns.
2. MCP and A2A specifications.
3. vLLM PagedAttention and SGLang serving architecture.
4. Agent evaluation frameworks and benchmark setup.
5. Calibration, selective prediction and abstention.
6. Small-model training with the Hugging Face course and nanoGPT-style implementations.

81.3 Security

1. Threat modeling and trust boundaries.
2. Least privilege, capability systems and zero trust.
3. OPA/Rego policy authoring.
4. Prompt injection and agent security benchmarks.
5. Supply-chain security, SBOMs, signing and provenance.
6. Secrets management and network egress control.

81.4 Research methodology

1. Define falsifiable hypotheses.
2. Establish transparent baselines.
3. Separate development and held-out environments.
4. Measure calibration and cost, not only accuracy.

5. Use ablations and paired comparisons.
6. Maintain experiment cards and immutable run manifests.

82. Suggested 12-Week MVP Learning/Build Schedule

Week	Systems focus	ML/product focus	Shared output
1	Rust/Tokio skeleton, state machine	Workload schema, mock agent	Architecture and threat model
2	PostgreSQL events	Model adapter interface	First admitted workload
3	Worker protocol	Python SDK	End-to-end mock run
4	Docker sandbox	Coding-agent loop	Shell action through broker
5	Budget ledger	Token/cost accounting	Budget exhaustion demo
6	Capability checks	Structured ActionProposal	Secret-read denial
7	Pause/cancel/retry	Reliability signals v0	Failure recovery demo
8	Artifact storage	Evidence generation	Patch and test evidence
9	API hardening	Dashboard process tree	Live observable run
10	Crash recovery	Multi-model adapter	Restart-safe demo
11	Integration and security tests	Benchmark harness	20+ scenarios
12	Packaging and docs	Demo narrative	Public MVP release candidate

83. First 30 Days Checklist

- [ ] Select a temporary codename and create mono-repository.
- [ ] Write six foundational ADRs.
- [ ] Define trust boundary and threat model.
- [ ] Freeze `AgentWorkload`, `AgentProcess`, `ActionProposal` and event v0 schemas.
- [ ] Implement in-memory lifecycle and invariants.
- [ ] Add PostgreSQL event append and materialized process view.
- [ ] Build deterministic mock model and shell tool.
- [ ] Launch commands inside a no-network Docker sandbox.
- [ ] Implement one file-read capability and one denial rule.
- [ ] Add token/time budget accounting.
- [ ] Produce a minimal EvidenceBundle.
- [ ] Build a CLI that runs and inspects one scenario.
- [ ] Create ten seeded repository tasks.
- [ ] Record every run in a dataset-ready schema.
- [ ] Demonstrate one blocked malicious instruction.

84. Architecture Decision Records to Write First

1. Why NEXUS-OS runs above Linux instead of replacing it.
2. Rust control plane and Python agent boundary.

3. Event sourcing with PostgreSQL.
4. Deterministic policy authority and OPA integration.
5. Sandboxing ladder and initial Docker limitations.
6. Canonical ActionProposal schema.
7. Model gateway neutrality and version recording.
8. Content-safe telemetry and privacy retention.
9. ContextPage and memory provenance model.
10. Kernel Model advisory-only guarantee.

85. Practical Learning Resources

Topic	Resource	How to use it
OS foundations	*Operating Systems: Three Easy Pieces*	Read targeted chapters alongside kernel features
Kernel implementation	MIT 6.S081	Use labs to understand process, traps, page tables and filesystems
Rust	The Rust Programming Language	Complete ownership, traits, error handling and concurrency chapters
Async Rust	Tokio tutorial	Implement worker and gateway concurrency
Linux isolation	Linux seccomp documentation	Build and test syscall profiles
Containers	gVisor documentation	Understand user-space kernel trade-offs
MicroVMs	Firecracker repository and docs	Prototype strong isolation and snapshots
Policy	OPA documentation and Rego playground	Write policy tests before integration
Workload identity	SPIFFE overview	Plan short-lived identities for distributed workers
Observability	OpenTelemetry specification	Define agent span and metric conventions
Tool protocol	MCP documentation	Implement adapters without bypassing the broker
Agent protocol	A2A specification	Design authenticated delegation
Agent OS research	AIOS paper and repository	Compare abstractions and experimental baselines
Memory	MemGPT, A-Mem and Memory OS papers	Design hierarchy and memory operations
Security	OWASP GenAI Security, AgentDojo	Seed threat scenarios and controls
Model serving	vLLM and SGLang papers/docs	Understand batching, KV cache and serving costs
Small LMs	Hugging Face LLM Course	Build tokenization and training fundamentals

Topic	Resource	How to use it
Minimal training	nanoGPT	Learn a transparent Transformer training stack
Open model data	Dolma and OLMo documentation	Study dataset curation and reproducibility
Benchmarks	SWE-bench, OSWorld, GAIA and AgentBench	Build evaluation adapters and understand limitations

86. What Not to Do

- Do not begin with a custom bare-metal kernel.
- Do not train a large model before collecting real trajectories.
- Do not expose shell access through a string-to-subprocess shortcut.
- Do not call a Docker container a complete security boundary.
- Do not store every prompt and secret in telemetry by default.
- Do not add ten agent frameworks before one workload works well.
- Do not let a model decide its own permissions.
- Do not publish security claims without attack benchmarks and false-positive results.
- Do not optimize benchmark scores by weakening policies silently.
- Do not build the dashboard before the event model is stable.
- Do not conflate chain-of-thought storage with evidence.
- Do not wait for the “perfect” architecture; maintain ADRs and iterate.

87. Immediate Recommended Scope

The best concrete starting product is:

> **NEXUS-OS Developer Preview: a governed runtime for coding agents that supports local/cloud routing, repository-scoped capabilities, sandboxed shell execution, budgets, approval gates, event-sourced replay and evidence export.**

The first serious research experiment should be model routing or early failure prediction because both can use trajectories generated by the MVP and have clear baselines. Context paging should follow once long-running tasks provide sufficient memory pressure. The Kernel Model should begin only after the schema and outcome labels have stabilized.

Appendices

Appendix A - MVP Acceptance Checklist

Runtime

- [ ] Workload manifest validates and admits.
- [ ] Root and child processes have stable identities.
- [ ] Lifecycle survives control-plane restart.

- [ ] Cancellation stops future actions and expires leases.

Security

- [ ] No action bypasses Tool Broker.
- [ ] Default-deny capability policy works.
- [ ] Repository secret paths are blocked.
- [ ] Network is disabled unless granted.
- [ ] Tool arguments are schema validated.
- [ ] Model cannot alter policy or grants.

Budgets

- [ ] Token, time and money limits are visible.
- [ ] Debits are atomic and attributable.
- [ ] Exhaustion produces safe partial completion.

Observability

- [ ] Process tree and action stream are visible.
- [ ] Model, tool and policy versions are recorded.
- [ ] Sensitive payloads are redacted or hashed.

Evidence

- [ ] Patch, commands and tests are connected.
- [ ] Final status distinguishes complete and partial.
- [ ] Evidence archive has integrity hashes.

Quality

- [ ] Deterministic mock suite passes.
- [ ] At least twenty integration scenarios pass.
- [ ] One prompt-injection scenario is blocked.
- [ ] Installation and demo are documented.

Appendix B - Experiment Card Template

Field	Description
Experiment ID	Stable identifier
Research question	The specific question being tested
Hypothesis	Directional, falsifiable claim
Primary metric	Metric chosen before running
Secondary metrics	Cost, latency, security, calibration and overhead
Dataset / benchmark	Version, split and environment digest
Models	Provider, model, date/version and decoding configuration
Policies	Policy bundle hash and capability profile

Field	Description
Baselines	Direct agent and transparent heuristic baselines
Intervention	Mechanism or model under test
Seeds / repetitions	Number and randomization strategy
Statistical method	Confidence interval or hypothesis test
Exclusions	Predefined invalid-run criteria
Results	Complete results including failures
Limitations	Threats to validity and generalization
Artifacts	Code, configs, logs, evidence and checkpoint references

Appendix C - Architecture Decision Record Template

TEXT

```
ADR-XXX: Decision title
Status: proposed | accepted | superseded
Date:
Owners:

Context
- What problem and constraints require a decision?

Decision
- What will be done?

Alternatives considered
- What other options were evaluated?

Consequences
- Positive, negative and operational effects.

Security and privacy impact
- Trust-boundary or data-handling changes.

Validation
- Tests, benchmarks or milestones that will confirm the decision.
```

Appendix D - Glossary

Term	Meaning
AgentProcess	Schedulable logical agent with identity, state, capabilities and budgets
AgentWorkload	Declarative user request and governance contract
ActionProposal	Typed request for a tool or external side effect
Capability	Narrow grant to perform an operation on a resource
ContextPage	Metadata-rich unit eligible for model context
EvidenceBundle	Integrity-linked record of a workload and its outputs

Term	Meaning
Kernel Model	Compact advisory model for control-plane predictions
Model Gateway	Normalized and credential-isolated model interface
Policy Engine	Deterministic service that authorizes or modifies proposals
Reliability Controller	Selects verification, routing, recovery or abstention actions
Sandbox profile	Declared execution isolation and resource configuration
Tool Broker	Sole mediated path to tool execution
Taint	Label indicating untrusted or sensitive influence
Trajectory	Ordered record of agent states, decisions, actions and outcomes

Appendix E - Comprehensive Reference List

1. Ge, Y. et al. **AIOS: LLM Agent Operating System.** arXiv:2403.16971, 2024. https://arxiv.org/abs/2403.16971
2. Packer, C. et al. **MemGPT: Towards LLMs as Operating Systems.** arXiv:2310.08560, 2023. https://arxiv.org/abs/2310.08560
3. Microsoft. **Windows for Agentic Experiences.** https://developer.microsoft.com/en-us/windows/agentic
4. NVIDIA. **NVIDIA Announces NemoClaw.** 2026. https://nvidianews.nvidia.com/news/nvidia-announces-nemoclaw
5. OpenAI. **The Next Evolution of the Agents SDK.** 2026. https://openai.com/index/the-next-evolution-of-the-agents-sdk/
6. Google Cloud. **Agent Development Kit.** https://docs.cloud.google.com/gemini-enterprise-agent-platform/build/adk
7. AGI Research. **AIOS Repository.** https://github.com/agiresearch/AIOS
8. All Hands AI. **OpenHands.** https://github.com/All-Hands-AI/OpenHands
9. Model Context Protocol. **Introduction and Specification.** https://modelcontextprotocol.io/docs/getting-started/intro
10. Agent2Agent Protocol. **A2A Specification.** https://a2a-protocol.org/latest/
11. OpenTelemetry. **Specification.** https://opentelemetry.io/docs/specs/otel/
12. Open Policy Agent. **Documentation.** https://www.openpolicyagent.org/docs
13. SPIFFE. **Overview.** https://spiffe.io/docs/latest/spiffe-about/overview/
14. Li, Y. et al. **Throughput-Optimal Scheduling for LLM Agents.** arXiv:2504.07347, 2025. https://arxiv.org/abs/2504.07347
15. **A Survey of Security Threats in LLM-based Agents.** arXiv:2506.23260, 2025. https://arxiv.org/abs/2506.23260
16. **Secure Design Patterns for LLM Agent Systems.** arXiv:2506.08837, 2025. https://arxiv.org/abs/2506.08837

17. Debenedetti, E. et al. **AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents.** arXiv:2406.13352, 2024. https://arxiv.org/abs/2406.13352
18. **AgentSecBench: Benchmarking Security of LLM Agents.** arXiv:2605.26269, 2026. https://arxiv.org/abs/2605.26269
19. OWASP. **Generative AI Security Project.** https://genai.owasp.org/
20. Linux Kernel Documentation. **Seccomp BPF.** https://docs.kernel.org/userspace-api/seccomp_filter.html
21. Google. **gVisor.** https://gvisor.dev/
22. Amazon Web Services. **Firecracker MicroVM.** https://github.com/firecracker-microvm/firecracker
23. Xu, W. et al. **A-Mem: Agentic Memory for LLM Agents.** arXiv:2502.12110, 2025. https://arxiv.org/abs/2502.12110
24. **Memory OS of AI Agent.** arXiv:2506.06326, 2025. https://arxiv.org/abs/2506.06326
25. **Agent-Native Memory: A Systems Study.** arXiv:2606.24775, 2026. https://arxiv.org/abs/2606.24775
26. Zhang, P. et al. **TinyLlama: An Open-Source Small Language Model.** arXiv:2401.02385, 2024. https://arxiv.org/abs/2401.02385
27. Hugging Face. **SmolLM2 Collection.** https://huggingface.co/collections/HuggingFaceTB/smollm2-6723884218bc2e6abbc5ed4a
28. Allen Institute for AI. **OLMo.** https://allenai.org/olmo
29. Soldaini, L. et al. **Dolma: An Open Corpus of Three Trillion Tokens for Language Model Pretraining Research.** arXiv:2402.00159, 2024. https://arxiv.org/abs/2402.00159
30. Jimenez, C. et al. **SWE-bench: Can Language Models Resolve Real-World GitHub Issues?** arXiv:2310.06770, 2023. https://arxiv.org/abs/2310.06770
31. SWE-bench. **Official Benchmark Site.** https://www.swebench.com/
32. Xie, T. et al. **OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments.** arXiv:2404.07972, 2024. https://arxiv.org/abs/2404.07972
33. **OSWorld 2.0.** arXiv:2606.29537, 2026. https://arxiv.org/abs/2606.29537
34. Liu, X. et al. **AgentBench: Evaluating LLMs as Agents.** arXiv:2308.03688, 2023. https://arxiv.org/abs/2308.03688
35. Mialon, G. et al. **GAIA: A Benchmark for General AI Assistants.** arXiv:2311.12983, 2023. https://arxiv.org/abs/2311.12983
36. Zhou, S. et al. **tau-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains.** arXiv:2406.12045, 2024. https://arxiv.org/abs/2406.12045
37. Xu, F. et al. **TheAgentCompany: Benchmarking LLM Agents on Consequential Real World Tasks.** arXiv:2412.14161, 2024. https://arxiv.org/abs/2412.14161
38. Wu, Q. et al. **AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation.** arXiv:2308.08155, 2023. https://arxiv.org/abs/2308.08155
39. LangChain. **LangGraph Documentation.** https://langchain-ai.github.io/langgraph/
40. Microsoft. **Semantic Kernel Documentation.** https://learn.microsoft.com/en-us/semantic-kernel/
41. CrewAI. **Documentation.** https://docs.crewai.com/
42. Kwon, W. et al. **Efficient Memory Management for Large Language Model Serving with PagedAttention.** arXiv:2309.06180, 2023. https://arxiv.org/abs/2309.06180
43. Zheng, L. et al. **SGLang: Efficient Execution of Structured Language Model Programs.** arXiv:2312.07104, 2023. https://arxiv.org/abs/2312.07104
44. OpenAI. **OpenAI Agents SDK - Python Documentation.** https://openai.github.io/openai-agents-python/

45. Google. **ADK Documentation.** https://google.github.io/adk-docs/
46. Bytecode Alliance. **WASI.** https://wasi.dev/
47. Arpaci-Dusseau, R. and Arpaci-Dusseau, A. **Operating Systems: Three Easy Pieces.** https://pages.cs.wisc.edu/~remzi/OSTEP/
48. MIT PDOS. **6.S081: Operating System Engineering.** https://pdos.csail.mit.edu/6.S081/
49. The Rust Project. **The Rust Programming Language.** https://doc.rust-lang.org/book/
50. Tokio. **Tokio Tutorial.** https://tokio.rs/tokio/tutorial
51. Hugging Face. **LLM Course.** https://huggingface.co/learn/llm-course/
52. Karpathy, A. **nanoGPT.** https://github.com/karpathy/nanoGPT
53. NIST. **AI Risk Management Framework.** https://www.nist.gov/itl/ai-risk-management-framework
54. SLSA. **Supply-chain Levels for Software Artifacts.** https://slsa.dev/
55. in-toto. **Software Supply Chain Integrity Framework.** https://in-toto.io/
56. Sigstore. **Keyless Signing for Software Artifacts.** https://www.sigstore.dev/

Closing Recommendation

Build NEXUS-OS as a sequence of increasingly powerful, measurable releases. The first goal is not to impress people with the number of boxes in an architecture diagram; it is to demonstrate that an autonomous coding agent can complete useful work while being denied actions it should never have been able to attempt. Once that runtime reliably generates high-quality trajectories, use the data to build routing, paging, recovery and Kernel Model research.

The strongest final identity for the project is:

> **A model-neutral, zero-trust operating layer for autonomous agents, combining capability security, resource-aware scheduling, integrity-aware memory, causal replay and a compact learned control plane.**

Even a well-executed subset—secure coding-agent runtime, event-sourced replay, multi-model routing and public benchmark—would be a substantial portfolio and research achievement. The full programme could become an open-source platform, a family of papers and a credible foundation for a startup or long-term research agenda.